

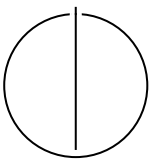
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Generation of CPU-GPU Heterogeneous Litmus Tests for Real Hardware

Nguyen Le





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

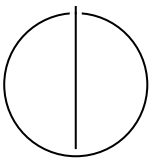
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Generation of CPU-GPU Heterogeneous Litmus Tests for Real Hardware

Generierung heterogener CPU-GPU-Litmus-Tests für reale Hardware

Author:	Nguyen Le
Examiner:	Prof. Dr. Pramod Bhatotia
Supervisor:	Dr. Anatole Lefort
Submission Date:	15.10.2026



Acknowledgements

I would like to thank Prof. Dr. Pramod Bhatotia for the opportunity to write this thesis at the Chair of Distributed Systems and Operating Systems, and my supervisor, Dr. Anatole Lefort, for the guidance and feedback throughout this thesis. During this thesis, I learned a great deal about weak memory models and heterogeneous CPU–GPU systems.

Abstract

Industry is rolling out hardware-coherent CPU–GPU systems, such as NVIDIA's GH200 Grace Hopper and AMD's Instinct MI300A, yet the memory-consistency guarantees across the two devices remain unclear. Goens et al. (PLDI 2023) propose compound memory models as a framework for reasoning about such systems, but their experiments run in the gem5 simulator and not on real hardware. Litmus-testing tools are well established for CPUs (e.g., herdtools7) and GPUs (e.g., GPUHarbor). Prior work has also generated and run heterogeneous CPU–GPU tests on hardware, but each test shape is authored by hand as a kernel or an execution graph rather than generated from a compact, one-line relaxation cycle within an established litmus toolchain. None of these tools emits HIP, AMD's GPU programming language.

This thesis addresses this gap with HetLitmus, an extension of herdtools7 that generates heterogeneous CPU–GPU litmus tests from relaxation cycles, compiles each test for an AArch64 or x86_64 CPU together with a CUDA or HIP GPU backend, and runs the CPU and GPU threads together under stress on both devices. Systematic enumeration of test shapes, thread placements, scopes and memory orders yields corpora of 10,979 tests across 29 shapes.

Run end to end on an NVIDIA DGX Spark, HetLitmus observed weak behaviour across the CPU and GPU. Of a 637-test evaluation subset, 193 showed their weak outcome, between 1 and 13,579 times in the 100,000 iterations of the run that observed it. With relaxed GPU accesses, the weak outcome was observed under every CPU order, including a full barrier. With the GPU writing and the CPU reading in message passing, it was observed under every GPU order at thread-block and device scope, but under no release or sequentially consistent order at system scope, the only scope that includes CPU threads. The source code is available at `https://github.com/lenguyenhcm325/herdtools7`.

Contents

Acknowledgements **iii**

Abstract **iv**

1 Introduction **1**

2 Background **4**
2.1 Heterogeneous architectures 4
2.2 Weak memory models 6
2.3 Scoped GPU memory models 8
2.4 Compound memory models 9
2.5 Litmus tests 10
2.6 herdtools7 11
2.6.1 Tool roles 11
2.6.2 Bell/LISA intermediate form 11

3 Related Work **12**
3.1 Test construction 12
3.2 Faithful test translation 13
3.3 Hardware execution and stress 14
3.4 Models and observations 15
3.5 Positioning of HetLitmus 16

4 Overview **17**
4.1 Design goals 17
4.2 Why herdtools7 18
4.3 Pipeline 18
4.4 User workflow 19

5 System Design **20**
5.1 Heterogeneous test generation 20
5.1.1 Per-device generation and merge 20
5.1.2 Merge soundness 21

5.1.3 Bell/LISA intermediate form . . . 22
5.1.4 Corpus enumeration . . . 23
5.2 Heterogeneous litmus format . . . 25
5.2.1 Per-thread device tags . . . 25
5.2.2 Two ISAs in litmus7 . . . 26
5.3 Harness emission . . . 27
5.3.1 Harness contents . . . 27
5.3.2 Annotation lowering . . . 28
5.3.3 Faithfulness checks . . . 29
5.4 Run environment . . . 30
5.4.1 Barriered persistent loop . . . 30
5.4.2 Stress and noise . . . 31
5.4.3 Tuning knobs . . . 33
5.4.4 Allocation . . . 33
5.5 Campaign and reporting . . . 35
5.5.1 Run outcomes and caveats . . . 36
5.5.2 Stop rule . . . 37

6 Evaluation **38**
6.1 Experimental setup . . . 38
6.1.1 Platform . . . 38
6.1.2 Evaluation set . . . 38
6.2 Campaign . . . 39
6.2.1 Stress tuning . . . 39
6.2.2 Passes . . . 40
6.2.3 Run validity . . . 41
6.3 Results . . . 41
6.3.1 Overview . . . 41
6.3.2 The full grid . . . 41

7 Conclusion **45**

8 Future Work **47**
8.1 Runs on AMD hardware . . . 47
8.2 Third vendor arm . . . 47
8.3 Positive control . . . 48

Abbreviations **49**

List of Figures **51**

List of Tables **52**

Bibliography **53**

1 Introduction

For most of their history, the central processing unit (CPU) and the graphics processing unit (GPU) of one machine were two distinct and separated memory worlds. The host copied its data to the device, launched a computation (kernel), and waited for it to complete before copying the results out and reading anything back [1]. However, that separation is no longer the only design: hardware-coherent CPU–GPU systems, such as NVIDIA's GH200 and GB10 as well as AMD's MI300A [2, 3, 1], are becoming increasingly common [4]. Six of the ten best-performing systems on the June 2026 TOP500 list, according to its benchmark, use such hardware [5, 2, 6, 1]. On such a machine, CPU and GPU threads can concurrently access shared memory using load and store operations, and software increasingly relies on this capability [7].

However, what each thread may observe from the other's writes to shared memory is less clear. Take message passing, the simplest such exchange: the CPU thread writes a register (message) and then raises a flag; the GPU thread sees the flag and reads the register (message). May it still read the old message? Each device answers only for itself. The CPU's model, for example total store order (TSO) on x86 or Arm's weak model, defines what the two stores promise to other CPU threads [8, 9]. On the GPU, synchronization operations have both a memory order and a scope; the scope limits which threads their ordering guarantees can cover [10, 11]. No model states what the pair promises. Software written for the pair assumes an answer nonetheless. A model taken as too strong costs performance, and one taken as too weak breaks the correctness of programs written against it [12].

Goens et al. [12] give the question a framework, the compound memory model: every device keeps the memory model of its own architecture, and the model of the fused machine is their composition. Their evidence, though, comes from a simulator, and the authors note themselves that a simulator need not show every behaviour a real machine does. The established way to put such a question to the hardware is litmus testing: a small concurrent program, run many times under stress, whose one interesting outcome is a behaviour the machine has if it is ever seen [13]. As Chapter §3 shows, litmus testing is mature for CPUs and GPUs in isolation, and prior work has also executed heterogeneous CPU–GPU tests for selected shapes. Existing work provides parts of this workflow, but no single approach combines systematic generation of heterogeneous tests with compilation and coordinated CPU–GPU hardware execution

while emitting the same tests as both Compute Unified Device Architecture (CUDA) and Heterogeneous-computing Interface for Portability (HIP) kernels. This thesis surveys prior work on memory models and litmus testing and develops HetLitmus, an extension of herdtools7, to investigate such cross-device behaviour on hardware.

In this thesis, we identify the following contributions:

1. heterogeneous test generation and exhaustively enumerated AArch64 and x86_64 corpora containing 10,979 tests across 29 shapes, device assignments, scopes and memory orders. (Section 5.1)
2. a new heterogeneous litmus format with one instruction set architecture (ISA) per column, a device tag on every thread and a GPU scope tree. (Section 5.2)
3. GPU code generation through dedicated vendor-specific back ends, including CUDA and HIP, behind a common architecture-independent front end. (Section 5.3)
4. a heterogeneous run environment with a per-iteration CPU–GPU rendezvous and stress on both devices and the interconnect, and a campaign that reports observations with run-validity information and caveats. (Section 5.4, Section 5.5)
5. validation of the methodology and of the test corpus on a physical NVIDIA machine, the DGX Spark. (Chapter §6)

The remainder of this thesis is structured as follows:

Chapter §2 provides background on heterogeneous architectures, weak memory models, scoped GPU memory models, the compound-memory-model framework, litmus tests and the relevant herdtools7 components. It provides the architectural and memory-model concepts and terminology used throughout the thesis.

Chapter §3 reviews prior work on litmus test generation, translation into executable code and hardware execution, together with formal and empirical studies of compound CPU–GPU behaviour. This comparison positions HetLitmus relative to prior work and identifies the gap that motivates its design.

Chapter §4 presents HetLitmus's design goals and its end-to-end user workflow at a high level. It follows a test from cycle-based generation through harness emission and device probing to a hardware campaign.

Chapter §5 presents the design and implementation of HetLitmus, which extends herdtools7 to generate, compile and run mixed-ISA CPU–GPU litmus tests.

Chapter §6 evaluates the methodology and corpus on the DGX Spark.

Chapter §7 summarizes HetLitmus's contributions and states the conclusions supported by the evaluation.

Chapter §8 outlines future work on AMD execution, broader vendor support and positive controls for non-observations.

2 Background

This chapter introduces the concepts used later: CPU–GPU memory architectures and coherent access (Section 2.1), weak-memory concepts and CPU models (Section 2.2), scoped GPU memory models (Section 2.3), the compound-memory-model framework (Section 2.4), litmus tests (Section 2.5), and the herdtools7 suite that HetLitmus extends (Section 2.6).

2.1 Heterogeneous architectures

Discrete memory and allocation. In the discrete design, the CPU and the GPU of one machine have two memories. The CPU with the memory attached to it is the host, the GPU with its memory the device [14]. Memory allocated on the device is accessible to the GPU only, and data moves between the two through explicit copy calls [15]. The programming model that goes with it manages device allocations and the transfers to and from the host by hand [16].

Three memory allocation modes appear again later. System-allocated memory is obtained through standard allocation functions such as `malloc`. On systems that support pageable-memory access, the GPU can access it directly. Managed memory is accessible to both processors, while system software manages its placement and any required migration. Pinned host memory stays in host memory but is page-locked and can be mapped for direct GPU access [17, 18].

These allocation modes do not by themselves determine whether concurrent access is coherent. With coarse-grained coherence, the devices' views are reconciled only at synchronization boundaries such as kernel completion, so one device need not see the other's updates while the kernel is running. Fine-grained coherence keeps updates visible while the CPU and GPU are both active. A CPU–GPU exchange during a kernel requires the latter [17, 19].

Hardware coherence. Hardware-coherent machines allow a CPU and a GPU to access shared memory through ordinary loads and stores without explicit copies [16, 1]. Their page-table organisation can vary: some use a logically combined CPU–GPU page table, while others maintain separate CPU and GPU page tables with synchronized entries

[20, 21]. This class of machine is what the thesis targets; NVIDIA's GH200 and GB10 and AMD's MI300A are the examples.

Concrete CPU–GPU systems. NVIDIA's GH200 Grace Hopper Superchip joins an Arm Grace CPU and a Hopper GPU over NVLink-C2C, a memory-coherent chip-to-chip interconnect [16]. It keeps two physical memories, up to 480 GB of LPDDR5X at the CPU and 96 GB of high-bandwidth memory (HBM)3 at the GPU, as two non-uniform memory access (NUMA) nodes of one machine [16, 15]. The two processors share one per-process page table, so a thread of either can access all system-allocated memory, and NVLink-C2C keeps the caches coherent at cache-line granularity without page migration [16].

The GB10 Grace Blackwell Superchip of the DGX Spark pairs a 20-core Grace CPU with a Blackwell GPU over the same NVLink-C2C, with one memory, 128 GB of LPDDR5x, coherent and shared by both [22, 3]. Its memory controller is in the CPU-side S-die, while the GPU reaches the shared LPDDR5X through the chip-to-chip link [23].

AMD's Instinct MI300A puts CPU chiplets, GPU chiplets and memory in one package, 24 Zen 4 cores and 228 compute units over an on-package Infinity Fabric [1]. It has one memory, 128 GB of HBM3 shared between CPU and GPU [24]. The CPUs are hardware-coherent with every CPU and GPU [1], and, unlike on the GH200, the GPU can access only a page table of its own, a copy of the system's entries that the Linux kernel's heterogeneous memory management keeps in sync [21].

Figure 2.1 compares the physical memory organisations of these three systems.

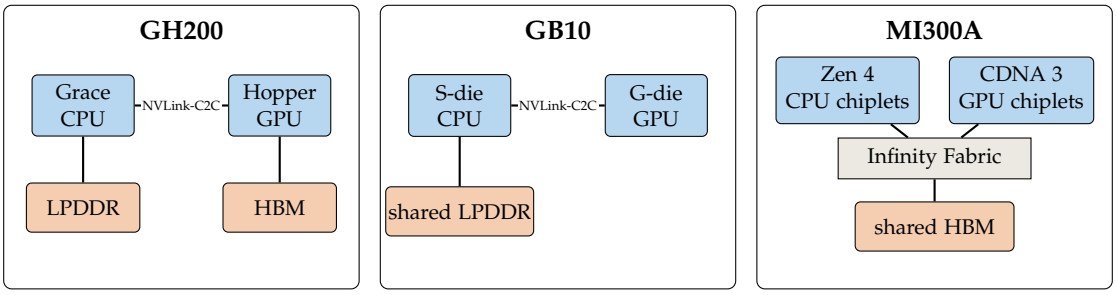


Figure 2.1: Simplified physical memory organisations of GH200, GB10 and MI300A [16, 23, 1].

CXL-attached devices. Beyond such packaged pairs, Compute Express Link (CXL) gives any accelerator the same attachment as an open standard: through CXL.cache

a device caches host memory under the MESI protocol at 64-byte lines, and through CXL.mem the host accesses device memory as cacheable memory [25].

GPU hierarchy and scope. On every one of these machines the GPU side is itself a hierarchy. An NVIDIA GPU is a collection of streaming multiprocessors (SMs), each with an L1 cache of its own, and one L2 shared by all of them. The threads of a kernel are grouped into thread blocks, called cooperative thread arrays (CTAs) in Parallel Thread Execution (PTX), and every thread of a block runs on one SM [10, 14]. An AMD GPU is organised alike, with workgroups dispatched to compute units [24]. Synchronisation across wider scopes can involve more of the memory hierarchy: between the threads of one block it can remain in the L1 they share, between the blocks of one GPU it can reach the L2, and between the GPU and the CPU it can reach memory [7]. That is why the memory models of GPUs attach a scope to every ordering.

These systems realise coherent CPU–GPU memory through markedly different memory-system designs, but neither the topology nor coherence alone determines which outcomes a concurrent program may produce. CPU and GPU instructions follow different ordering rules, and GPU orderings are also scoped, so correctness depends on how those rules interact across the device boundary. The following sections introduce the CPU and GPU memory models and GPU scopes. They then present the compound-memory-model framework used to describe memory ordering across the device boundary.

2.2 Weak memory models

A memory consistency model determines which values a load may return in a multithreaded program [26]. The model every other is measured against is sequential consistency (SC) [27]: the result of any execution is the same as if the operations of all processors were executed in some sequential order, and the operations of each processor appear in that sequence in the order given by its program [28]. Many real multiprocessors implement weak, or relaxed, memory models. Hardware optimizations such as store buffers, cache hierarchies, and speculation leave sequential program behavior unchanged but can cause concurrent threads to observe different views of shared memory [8].

CPU models relax SC to different degrees; two examples are x86-TSO and AArch64. x86-TSO [8] is a store-buffer model: every processor has a FIFO buffer of pending writes, a read returns the processor's own buffered write to that location if there is one, the only reordering is a read passing an earlier write to another location, and MFENCE flushes the buffer. In the store buffering (SB) test two processors each write one location

and then read the other's; under x86-TSO both may read the old value. Compared with x86-TSO, AArch64 permits all four load/store reorderings between independent ordinary accesses to different locations [29]. Selected orders can be restored with DMB barriers or acquire and release instructions.

A release store orders the thread's earlier memory accesses before the store. An acquire load orders the thread's later memory accesses after the load. When an acquire load in another thread reads the value written by the release store, accesses before the release in the writing thread are ordered before accesses after the acquire in the reading thread [30].

A small producer–consumer example illustrates the difference. A producer writes the data `x=1` and then the flag `y=1`; a consumer reads `y` and then `x`. This four-access pattern is called message passing (MP); its litmus-test representation is introduced in Section 2.5. Its outcome of interest, `y=1, x=0`, means that the consumer sees the flag but still reads the old data. Table 2.1 compares this outcome under SC, x86-TSO and AArch64.

Table 2.1: The MP outcome under SC, x86-TSO and AArch64.

Model and variant	Accesses to `y`	`y=1, x=0`	Reason
SC, plain	plain	forbidden	SC preserves the producer's store–store order and the consumer's load–load order [28].
x86-TSO, plain	plain	forbidden	x86-TSO preserves the producer's store–store order and the consumer's load–load order [8].
AArch64, plain	plain	allowed	Plain accesses do not preserve both required program-order edges [30].
AArch64, release/acquire	Producer: store-release (`STLR`); consumer: load-acquire (`LDAR`)	forbidden	`STLR` orders the producer's earlier write `x=1` before its flag store `y=1`. `LDAR` orders the consumer's later read of `x` after its flag load. If `LDAR` reads the value written by `STLR`, these orders forbid `x=0` [30].

Alternatively, AArch64 forbids the outcome with `DMB ST` between the writes and `DMB LD` between the reads [9].

Memory models can be stated axiomatically in the cat language [31]. An execution is a set of events, the reads, writes and fences the instructions produce, with relations

over them: program order `po` within a thread; reads-from `rf`, from a write to a read that takes its value; coherence `co`, a total order over the writes to one location; and from-read `fr`, from a read to every write coherence-after the one it read [32]. A model states which combinations of these relations must be acyclic or irreflexive, and rejects candidate executions that violate those constraints. SC is the acyclicity of all four together. TSO keeps only part of `po`: a write followed by a read is no longer an edge, unless an MFENCE stands between them [32]. A relaxation is an edge the model does not preserve, so that a cycle through it is allowed, and a litmus test (Section 2.5) exercises one such cycle.

2.3 Scoped GPU memory models

The GPU models of both vendors HetLitmus targets, NVIDIA's PTX model [10] and AMD's [11], have one shape: a synchronising memory operation carries a memory order and a scope, and so does a fence. The vocabulary is the same on both sides up to its spelling, which is what lets the GPU thread of a heterogeneous litmus test be written once and lowered to either vendor (Section 5.2).

The scope names the set of threads an operation's ordering reaches. Table 2.2 summarises the three levels common to both vendors [10, 11].

Table 2.2: The GPU scope levels common to PTX and AMD.

Level	PTX	AMD	Threads included
Thread block	`.cta`	`workgroup`	One thread block or workgroup
Device	`.gpu`	`agent`	Threads on one GPU
System	`.sys`	`system`	GPU and host threads in the system

An ordering holds only between threads included in its scope. A GPU operation can therefore participate in an acquire–release exchange with a CPU thread only at system scope.

The memory order determines which surrounding accesses are ordered. Table 2.3 summarises the five order categories used by HetLitmus for both vendors [10, 11].

Table 2.3: The GPU memory orders and their effects.

Order	Effect
`relaxed`	Does not order surrounding accesses
`acquire`	Orders later accesses after it
`release`	Orders earlier accesses before it
`acq_rel`	Combines acquire and release ordering
`sc`*	Adds a total-order constraint at the chosen scope

* PTX provides `fence.sc`, while AMD supports both `seq_cst` atomics and fences.

The lowering of these orders and scopes to CUDA and HIP operations is described in Subsection 5.3.2 and summarised in Table 5.2.

2.4 Compound memory models

Each model of Section 2.2 and Section 2.3 describes one device. Goens et al. [12] therefore ask what memory model results when devices with different models are joined by an inter-device interconnect. Their compound model is compositional: each device retains the ordering rules of its own model.

Their Local Orderings by Stalling on Threads – Partial-Order Propagation (LOST-POP) operational model explains this composition through propagation. Each read or write begins as a request at its issuing thread, and the model records which other threads it has reached. For a write, reaching a thread means that its value is visible there. If an architecture requires one request to precede another, the later request cannot reach a thread before the earlier one. A release store, for example, cannot reach a consumer before a write ordered ahead of it within the release's scope. Fences enforce order by stalling later requests rather than by propagating themselves [12].

Every ordering constraint carries a scope, so its stall applies only to threads in that scope. Constraints from a model without scopes, such as a CPU model, are treated as system-scoped. The propagation mechanism is shared, while each thread applies the rules of its own architecture; this is what allows different device models to be composed [12].

An axiomatic model is provided only for the x86-TSO+PTX instantiation. It is designed to be a sound abstraction of the x86-TSO+PTX operational model and is used to prove a scoped-C/C++ compiler mapping [12]. It does not provide architecture-independent axioms that can be applied directly to other combinations, such as AArch64 with PTX. For another architecture pair, new cross-device relations and axioms determining which executions are allowed would have to be defined and related to that

pair's LOST-POP instantiation. This is separate formal-modeling work and lies outside this thesis. HetLitmus therefore uses the framework to motivate cross-device litmus experiments but reports hardware observations without assigning model-derived allowed or forbidden verdicts.

2.5 Litmus tests

A *litmus test* is a small parallel program whose purpose is to exercise the memory model of a parallel, shared-memory computer. A source has three main sections: the initial state fixes the initial values of registers and memory locations, the code section gives the code that runs concurrently, and the final condition applies to the final values of registers and memory locations [31].

Each litmus-test shape isolates a specific weak-memory behaviour, and its final condition identifies an outcome that would demonstrate that behaviour on hardware. For example, in an independent reads of independent writes (IRIW) variant, two threads write separate locations and two other threads read both, with each reader's load order preserved. If one reader sees `x=1, y=0` while the other sees `y=1, x=0`, each write is visible to one reader but not the other. This shows that independent writes need not become visible to all threads in one common order [33].

Across a suite of shapes, the observed outcomes provide an empirical characterization of the weak-memory behaviours exhibited by a machine without first requiring a complete axiomatic model. Where such a model exists, its allowed outcomes can be compared with the observations. Failure to observe an outcome does not establish that it is impossible [32].

Shapes that are commonly used are named by convention, for example, message passing (MP), store buffering (SB), load buffering (LB), write-to-read causality (WRC), read-to-write causality (RWC) and IRIW [32]. Herding Cats also names five same-location patterns, coWW, coRW1, coRW2, coWR and coRR [32]. Table 5.1 lists every shape the thesis uses with its cycle.

Listing 2.1 gives the plain AArch64 form of the MP pattern introduced in Table 2.1, taken from `catalogue/aarch64/tests/MP.litmus` as herdtools7 ships it [34], with its metadata lines omitted. The first line names the architecture and the test; the second records the cycle from which the test was generated (Subsection 2.6.1), `PodWW Rfe PodRR Fre`: P0's two writes in program order, P1's read of the flag from P0's write, P1's two reads in program order, and the from-read edge from P1's data read to P0's write, whose value it did not see. The final condition expresses this outcome using P1's registers, `1:X0=1` and `1:X2=0`.

```
AArch64 MP
"PodWW Rfe PodRR Fre"
{
0:X1=x; 0:X3=y;
1:X1=y; 1:X3=x;
}
 P0          | P1          ;
 MOV W0,#1   | LDR W0,[X1] ;
 STR W0,[X1] | LDR W2,[X3] ;
 MOV W2,#1   |             ;
 STR W2,[X3] |             ;
exists (1:X0=1 /\ 1:X2=0)
```

Listing 2.1: The AArch64 MP test shipped with herdtools7, with metadata omitted.

2.6 herdtools7

herdtools7 is the tool suite HetLitmus extends. It bundles a generator, a runner and a simulator: diyone7, litmus7 and herd7 [34].

2.6.1 Tool roles

diyone7 generates a litmus test from one cycle of relaxations, such as that of Listing 2.1, for the architecture named on its command line. litmus7 compiles and runs litmus tests on hardware. herd7 simulates a litmus test against an axiomatic memory model written in cat (Section 2.2) [31].

2.6.2 Bell/LISA intermediate form

herdtools7's generic architecture is Litmus Instruction Set Architecture (LISA), a deliberately minimal assembly language with reads, writes, branches, fences and read-modify-writes, intended for designing consistency models of weakly consistent systems [35]. LISA instructions can carry tags, such as C++ release and acquire annotations; their memory-model meaning is defined by a `cat` specification [35]. Scopes are special tags whose levels and hierarchy are defined in a Bell file; a `scopes:` declaration in the litmus-test body places threads in that hierarchy [35, 31]. For example, with the levels `sys`, `gpu` and `cta`, `scopes: (sys (gpu (cta P0) (cta P1)))` puts P0 and P1 in two different CTAs of the same GPU.

The Bell file also declares the available instruction tags and specifies which may annotate reads, writes and fences (Listing 5.2 is the one HetLitmus uses). herd7 loads it ahead of a cat model, and diyone7 given a bell file generates LISA litmus tests whose annotations are drawn from it [31].

3 Related Work

To position HetLitmus, this chapter reviews prior work on systematic construction of heterogeneous test families from relation cycles (Section 3.1); faithful translation to the target CPU, CUDA and HIP backends (Section 3.2); coordinated CPU–GPU hardware execution with stress (Section 3.3); and the relationship between formal-model verdicts and finite hardware observations (Section 3.4). The final section brings together the tooling comparisons and explains how HetLitmus addresses the remaining gap (Section 3.5).

3.1 Test construction

As described in Subsection 2.6.1, diyone7 constructs a litmus test from a relation cycle representing a candidate weak execution. A cycle is independent of register names, memory-location names and instruction syntax, making it a compact description of a test family.

Alglave et al. [36] retain the relation cycle as the test shape and add two separate fields to the generated GPU-only test: a scope tree places threads in the GPU hierarchy, and a memory map assigns locations to shared or global memory. The tool uses the scope tree to place testing threads in the specified CTA arrangement. Both upstream diyone7 and this GPU extension generate single-architecture tests, so neither can assign a test's threads across a CPU and GPU.

Srivastava [37] instead require a more detailed cyclic execution graph: the user supplies each read and write, the relations between them and a computation mapping that assigns every thread to the CPU or GPU. Bagchi et al. [7] use this framework for 1,960 heterogeneous MP variants that vary the assignment of producer and consumer roles, memory orders, fences and GPU scopes. The separate `hetero-cuda-litmus` repository encodes 23 test families as hand-written CUDA kernels. Each kernel fixes the test structure, while scripts generate variants by enumerating device assignments, scopes, memory orders and fences [38]. These approaches vary heterogeneous configurations around a supplied structure, but adding a shape requires authoring another event-level graph or kernel rather than supplying a compact relation cycle.

Model-directed systems derive tests from formal specifications. Lustig et al. [39] use Alloy to synthesize minimal tests from axiomatic models. They aim to exclude

over-synchronized tests in which removing or weakening a fence or memory-order annotation would leave the same outcomes allowed. Memalloy uses bounded searches to construct litmus tests that distinguish memory models [33]. Iorga et al. [40] use the Alloy Analyzer to generate tests from an axiomatic model of Intel Xeon–field-programmable gate array (FPGA) shared-memory semantics. These methods provide model-specific coverage or tests that expose differences between models, but require the relevant axiomatic model or compiler mapping to be defined first. As Section 2.4 explains, an axiomatic compound model is available for x86-TSO+PTX, but model-directed generation cannot be applied to the other architecture pairs until corresponding models have been established.

3.2 Faithful test translation

For HetLitmus, faithful translation means that every source operation is translated into code with its specified kind, memory order and, where applicable, scope. No operation may be omitted or reordered.

After a test has been selected, Srivastava's execution graph serves as a backend-neutral input. A separate JSON configuration selects the backend, CPU memory order, execution method and stress, and templates emit C++, CUDA or Vulkan without requiring the graph to be rewritten [37]. GPUHarbor likewise stores thread actions and possible outcomes in a neutral JSON description. LitGen emits WGSL (WebGPU Shading Language) for WebGPU compute shaders, which are GPU programs analogous to CUDA kernels. For Vulkan, it emits OpenCL source that Clspv compiles to SPIR-V, the binary shader representation accepted by Vulkan [41]. Both systems separate test descriptions from backend-specific code generation, but neither paper describes an operation-by-operation check of the generated code against those descriptions.

Alglave et al. [36] compile their GPU-specific `.litmus` tests into a host program and a PTX or OpenCL kernel. The general `.litmus` format remained, but the `GPU_PTX` and OpenCL parsers and model support were removed from herdtools7 in June 2015.[1] They inspect the compiled code for every test they execute, checking that its memory operations have not been removed or reordered. This verifies the generated programs used in their campaign, but not the translation backend as a whole.

Memalloy instead formalises source and target memory models and a candidate compiler mapping in Alloy. It searches for a litmus test whose outcome is forbidden by the source model but permitted for the translated test by the target model; this is a counterexample to the mapping. For example, it finds a flaw in an OpenCL-to-AMD-style GPU mapping [33]. This checks the compiler mapping's semantics, not whether

[1] herdtools7 commit `7ea48355a`

generated test code matches its input description.

3.3 Hardware execution and stress

Whether a finite hardware run exposes a weak outcome depends on the test's placement, surrounding memory traffic, and iteration schedule [42, 43]. The original `litmus` runner targets placement and memory traffic: it executes multiple test copies in parallel and varies role-to-thread assignments, processor affinity, direct or shuffled location access, memory preload and iteration synchronization. These parameters can substantially change the frequency of interesting outcomes [13].

PerpLE instead targets the iteration schedule. Conventional iterative tests synchronize the threads before each iteration so that the corresponding test bodies overlap. PerpLE synchronizes only at launch and lets each thread loop independently. It replaces a store of value a to location m in iteration n_t with a store of $k_m n_t + a$, where k_m is the number of distinct integer values stored to m in the original test. For a non-initial loaded value v, assuming distinct residues for the original stored values, the remainder modulo k_m identifies a, and dividing $v - a$ by k_m recovers the producing iteration; this allows interactions between different loop iterations to be reconstructed. Removing the repeated synchronization increases the opportunities to observe outcomes per unit time [43]. The original `litmus` runner and PerpLE both increase opportunities to observe weak outcomes in CPU-only hardware tests, but neither coordinates one test across a CPU and GPU.

Srivastava's compiler applies both iteration schemes to heterogeneous tests. Its single-instance mode relaunches the test threads for each iteration and checks that iteration's final state; its perpetual mode launches them once and uses observer traces to infer cross-iteration behaviour [37]. Across six two-thread shapes on three integrated systems, both modes completed on an NVIDIA Jetson Orin Nano. On the tested AMD Ryzen and Intel systems, the CPU stopped observing the GPU's updates to a shared Vulkan allocation after a few iterations, preventing a reliable experiment.

GPU execution environments add work around the litmus test to increase contention or align its participants. In the harness of Alglave et al. [36], non-testing threads repeatedly access disjoint locations to stress the memory system, lanes contend for shared-memory banks, thread identifiers are randomized and testing threads synchronize immediately before the test. The effect of each mechanism varies by test and device, so the harness evaluates combinations rather than assuming one setting is universally effective.

Later work treats the selection of these parameters as a tuning problem. Sorensen et al. [44] use litmus test microbenchmarks to choose a memory-access sequence and

amount of stress for each GPU before applying the resulting configuration to CUDA applications. Kirkham et al. [45] study 1,000 stress configurations on GPUs from three vendors and develop methods for selecting configurations that generalize across tests and devices. MC Mutants provides an explicit measure of effectiveness: it modifies a test so that an outcome related to a potential bug is allowed, then scores an environment by whether and how quickly it observes that outcome [42]. GPUHarbor applies this method to WebGPU and Vulkan, using each device's weak-behaviour profile to select stress configurations for related conformance tests [41].

These GPU execution environments improve stress generation and selection, but their test participants remain on the GPU. They therefore provide neither mixed CPU–GPU coordination nor stress directed at the cross-device path.

Among systems that execute mixed tests, the `hetero-cuda-litmus` implementation is the closest comparison to HetLitmus: it natively executes mixed CPU–GPU tests in CUDA, coordinates the participants per instance and supports stress on both devices [38]. It is specific to NVIDIA systems and CUDA; it provides neither a HIP backend for AMD GPUs nor a backend for other vendors.

3.4 Models and observations

Assigning allowed or forbidden verdicts to heterogeneous executions requires a formal compound model for the architecture pair. Constructing its axiomatic semantics is substantial formal work; without it, hardware testing reports only the outcomes observed in finite runs.

Goens et al. [12] formalize compound memory models but do not evaluate them on physical heterogeneous hardware. As described in Section 2.4, their axiomatic result is specific to x86-TSO+PTX, while their empirical evaluation runs x86+GCN3 litmus tests in the gem5 simulator. The authors caution that the simulator may omit behaviours of a real machine. Their work therefore motivates cross-device hardware testing as a complementary empirical step.

The Grace Hopper study reports weak outcomes when scope or ordering was insufficient and none in its correctly synchronized system-scope MP variants [7]. The authors interpret this agreement with their expected Arm+PTX behaviour as validation that Grace Hopper enforces the compound model and as evidence supporting independent compilation of CPU and GPU code. However, agreement on these MP variants does not validate the compound model beyond the tested cases.

3.5 Positioning of HetLitmus

The closest systems each provide a different part of the required workflow, but none covers it end to end. Alglave et al. [36] generate cycle-based scoped GPU tests and inspect the compiled code, but their tests remain GPU-only. Srivastava [37] compile heterogeneous tests from a backend-neutral representation, but each cyclic execution graph must be supplied by the user. Of the prior systems, Heterogeneous CUDA Litmus is closest to HetLitmus's workflow: it coordinates and stresses CPU and GPU participants and enumerates variants of hand-written CUDA test families by sweeping device assignments, scopes, memory orders and fences, but does not generate families from relation cycles or provide a HIP backend [38].

HetLitmus addresses these gaps within diy7 and litmus7. It generates heterogeneous test families from relation cycles, compiles each test for AArch64 or x86_64 together with CUDA or HIP, checks the translated operations, and runs the CPU and GPU participants in one stressed harness. Its 10,979-test corpus systematically enumerates a deliberately bounded space of 29 shapes and selected device assignments, scopes and memory orders.

4 Overview

This chapter gives an overview of HetLitmus: the design goals behind it (Section 4.1), why herdtools7 is the suite it extends (Section 4.2) and the pipeline from a relaxation specification to a results table at a high level (Section 4.3). It closes with how a user drives the whole workflow with a few commands, from probing the device to the campaign's results table (Section 4.4). Chapter §5 gives the technical detail.

4.1 Design goals

The gaps summarized in Section 3.5 motivate the following design goals for HetLitmus. Chapter §5 describes how the system addresses them.

Systematic coverage. Tests derive from relaxation cycles, and a family is the exhaustive enumeration of a defined space: shape, device assignment, scope and the memory order on each side. A hand-written suite is limited to what was written: a new order or scope means new tests by hand, where a family gains one value on an axis.

Per-device semantics. Every thread of a heterogeneous test is generated and lowered under the model and vocabulary of its own architecture; no new vocabulary is defined for the pair. This turns the premise of the compound model, that every device keeps its own model (Section 2.4), into a construction rule.

Vendor independence. The GPU column is written in a generic scoped vocabulary, not in one vendor's, so one test runs unchanged on NVIDIA and AMD; a vendor is one emitter, deciding which CUDA or HIP operation each annotation becomes.

Faithfulness. The program that runs carries exactly the orders, scopes and operation kinds of the test, and nothing around them adds an ordering of its own; both are verified by reading the translation back. Without it the translation may strengthen, weaken or misplace an annotation, and an observation then reflects the program that ran rather than the test.

Raising the chance of observation. A weak outcome across two devices is rare, so the run environment is built to raise its chance: the two devices race in every iteration, the shared locations lie in memory both access coherently, and memory stress and noise on either side push the accesses into the timings a weak outcome needs.

4.2 Why herdtools7

herdtools7 was chosen because most of the pipeline already exists there. diyone7 generates litmus tests from cycles of relaxations, litmus7 compiles and runs them on CPUs of several ISAs, and both read one format, so the file diyone7 writes is the file litmus7 runs. Both tools are parameterised by the architecture: every ISA is a module with its own parser and code emitter, so a heterogeneous machine is one more such module (Subsection 5.2.2). Its generic architecture, LISA with a bell file, gives a GPU front end without a new parser (Subsection 5.1.3). It is open source under the CeCILL-B licence and has been developed for over a decade. As of 13 September 2026, its upstream repository had 1,377 commits in the preceding year and 312 GitHub stars [34].

4.3 Pipeline

Figure 4.1 shows the pipeline from a specification to a results table in comma-separated values (CSV) form.

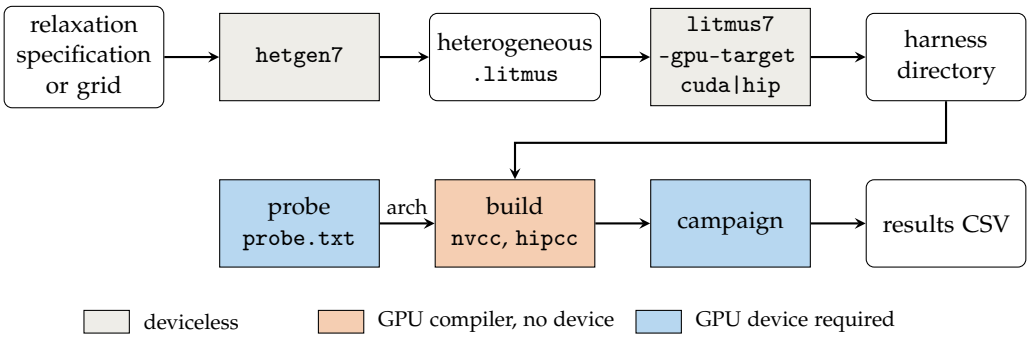


Figure 4.1: The HetLitmus pipeline. Rounded boxes are artefacts, rectangular boxes are steps, and the fill of a step says what it needs.

Generation. The input is one annotated cycle per device over one shape, together with the bell file that declares the GPU vocabulary. `hetgen7` runs the diy engine once

per device and merges the two results into one heterogeneous `.litmus` test.

Emission. `litmus7` parses each column with the parser of that column's device and emits one harness directory per test: the GPU kernel with its host driver in the vendor language selected by `-gpu-target`, the CPU threads as C around litmus7's own inline assembly, the runtime headers and the build files.

Probe and build. The probe records the device architecture and its attributes as key-value lines, such as `suggested_cuda_arch=sm_121` and `cooperativeLaunch=1`, of which the build reads the architecture. The build compiles and links every harness directory with the vendor compiler for that architecture.

Campaign. The campaign runs each harness binary until a stop rule assigns it a terminal state, re-invoking it with fresh seeds, writes one row per test to the state CSV and prints one line per test and a summary on standard output. Optionally, a tuner searches the stress knobs on the device beforehand for a higher observation rate (Subsection 5.4.3).

4.4 User workflow

Listing 4.1 is the minimal device run for an AArch64 CPU and a CUDA GPU. The probe runs first, then the corpora are generated as build products (Subsection 5.1.4).

```
export RESULTS=hetlitmus/run-out/<tag>       # every artefact of this run goes here
sh hetlitmus/probe-cuda.sh                   # record the device architecture and attributes
make hetlitmus-corpus-gen                    # generate the corpora
# emit one CUDA harness directory per test
hetlitmus/emit-het.sh --gpu-target cuda \
    <corpus_dir>
hetlitmus/build.sh $RESULTS/emit             # compile and link every harness for that device
# run each harness, one CSV row per test
python3 hetlitmus/campaign.py \
    --corpus $RESULTS/emit \
    --state $RESULTS/campaign.csv
```

Listing 4.1: One device run, AArch64 CPU and CUDA GPU

5 System Design

This chapter follows a test through HetLitmus, the design of each stage and its realisation inside herdtools7: generation (Section 5.1), the heterogeneous format (Section 5.2), harness emission (Section 5.3), the run environment (Section 5.4) and the campaign (Section 5.5). Although the implementation is specific to herdtools7, the concepts behind each stage are transferable to other toolchains.

5.1 Heterogeneous test generation

HetLitmus uses the relation cycle introduced in Section 2.5 as a compact, declarative input. It generates variants by attaching memory-order, scope and barrier annotations to the edges of one ISA-independent cycle.

The diy cycle engine, `gen/top_gen.ml`, turns one cycle into a test in which every processor uses the same architecture, so heterogeneous generation requires a GPU front end the engine can drive and a way to combine two architecture-specific runs.

5.1.1 Per-device generation and merge

`hetgen7` is a driver that runs the diy engine once per device and merges the columns (Figure 5.1). The CPU run is instantiated over AArch64 or x86_64, selected by `-cpu-arch`; the GPU run is fixed to the Bell/LISA builder. Each run demands exactly one cycle. The two runs hand their results to the merge in one common form, the test as text: its initial-state atoms, the instruction cells of each processor and its condition atoms, each already printed as the string it will have in the `.litmus` file. Producing that form is a new member of the diy builder signature, implemented once in the engine so that no per-architecture code is needed.

The merge is keyed by `-devices`: each processor's cells, init atoms and condition atoms come from the run that owns its device; global init atoms are unioned and deduplicated, global condition atoms unioned. `hetgen7` also composes the scope tree, one CTA per GPU processor: scope is an axis of the corpus (Subsection 5.1.4), and if two tested threads shared a CTA the `cta` and `gpu` cells would be the same experiment for that pair. Its own flags are `-cpu-arch`, the CPU ISA, `aarch64` or `x86_64`; `-cpu` and `-gpu`, the edge cycle of each device; `-devices`, the comma-separated device of each

processor in order; and `-com`, the test's comment line. Listing 5.1 is the invocation with which the corpus generates `MP-cg-sys-plain.acq`, the reference test of this chapter.

```
hetgen7 -set-libdir $HERDLIB -bell hetlitmus/bells/gpu.bell -oneloc \
  -cpu-arch aarch64 -devices cpu,gpu -name MP-cg-sys-plain.acq \
  -cpu "PodWW Rfe PodRR Fre" \
  -gpu "PodWWRelaxedSysRelaxedSys RfeRelaxedSysAcquireSys PodRRAcquireSysAcquireSys
      FreAcquireSysRelaxedSys"
```

Listing 5.1: The invocation that generates the reference test `MP-cg-sys-plain.acq`: CPU writer, GPU reader with acquire reads at system scope.

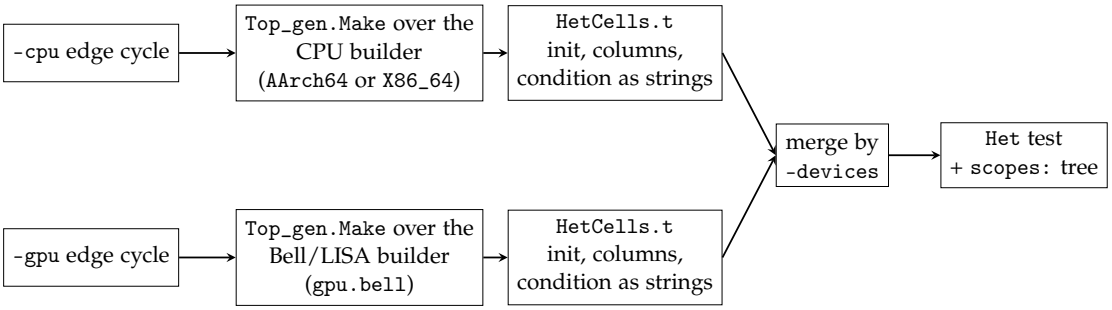


Figure 5.1: Two engine runs, erased to strings and merged by device assignment.

5.1.2 Merge soundness

`hetgen7` checks two things before it merges. Position by position, the two parsed cycles must agree on an annotation-stripped edge skeleton: the edge kind, `Po`, `Rf`, `Fr` or `Co`, with a fenced program-order edge such as `DMB.SYdWW` counted as the plain `PodWW`; then whether the two accesses hit the same location or different ones, `Pos` against `Pod`; whether they lie on one processor or on two, `Rfi` against `Rfe`; and the directions of the two accesses, `PodWR` against `PodRW`. Both runs must then produce one processor per device `-devices` lists.

For the cycles used in the corpus, equal shape is what makes the union meaningful. A diy test is determined by its cycle, which fixes the processor count, the accesses on each processor, the shared locations and the final condition over the reader registers; only the instruction encoding is architecture-specific. Two runs over the same skeleton therefore agree on the structure and differ only per processor, in the spelling of each cell, `STR W0,[X1]` against `w[release,sys] x 1`, and in the register names of its init and condition atoms.

5.1.3 Bell/LISA intermediate form

Adding a native herdtools7 architecture for each GPU target would require a lexer (the pass that cuts the source text into tokens), a parser, a base module and a generator backend for each one. HetLitmus instead writes the GPU side once in LISA (Subsection 2.6.2) under its own bell file, Listing 5.2. The file declares five memory-order tags, the three-level scope hierarchy `cta < gpu < sys`, and the tags admitted on reads, writes and fences. The scope names follow PTX spelling, but serve as HetLitmus's common intermediate vocabulary; the HIP emitter maps them to work-group, agent and system scope. The tags carry no meaning of their own; what each order and scope becomes on a device is decided by that vendor's emitter (Subsection 5.3.2). The permitted memory-order annotations for reads, writes and fences are adapted from upstream's `herd/libdir/c11.bell`, with every annotated operation also carrying a scope; the `narrower` and `wider` scope functions are adapted from `catalogue/tutorial/bells/jaguar.bell`.

```
"Scoped GPU Bell annotations (HetLitmus)"

enum memory_order = 'relaxed || 'acquire || 'release || 'acqrel || 'sc
enum scopes = 'cta || 'gpu || 'sys

let narrower(s) = match s with
  || 'sys -> 'gpu
  || 'gpu -> 'cta
end

let wider(s) = match s with
  || 'gpu -> 'sys
  || 'cta -> 'gpu
end

instructions R[{'relaxed,'acquire,'sc}, scopes]
instructions W[{'relaxed,'release,'sc}, scopes]
instructions F[{'acquire,'release,'acqrel,'sc}, scopes]
```

Listing 5.2: The HetLitmus bell file `hetlitmus/bells/gpu.bell`.

The bell also fixes how diy spells an annotated edge: each tag is capitalised and appended to the edge, so a program-order edge from a relaxed write to a release write at system scope reads `PodWWRelaxedSysReleaseSys`, and a fence edge carries the fence's own tag and scope in front, `FenceAcqrelSysdWWRelaxedSysRelaxedSys` for an acquire-release fence between two relaxed writes to different locations.

5.1.4 Corpus enumeration

The corpus is enumerated by `hetlitmus/tests/grid.py`, which runs `hetgen7` once per cell of a five-axis grid: shape, device cut, scope, CPU order and GPU order. The three corpora, heterogeneous over AArch64, heterogeneous over x86_64 and GPU-only, are build products rebuilt whenever the grid, the bell or a generator changes.

The shapes are the 29 of Table 5.1. A device cut assigns each processor to a device; every assignment but all-CPU and all-GPU is a cell, reduced by symmetry: when the processors of a cycle can be renamed without changing it, assigning the devices before or after that renaming gives the same test, and only one of the two assignments is kept. SB thus has the one cut `cg`, since `gc` is the same test with its two processors swapped, while MP, whose writer and reader differ, has both. The all-GPU assignment is the GPU-only corpus, generated through `diyone7` with an explicit scope tree, and an all-CPU assignment is an ordinary single-ISA test, which upstream herdtools7 already generates and runs.

The GPU order annotates every access of the GPU columns alike, at the cell's scope: `rlx` leaves them relaxed, `acq` tags the reads acquire, `rel` the writes release, `ra` both, and `sc` tags every access sequentially consistent. Four further orders, `fsc`, `facq`, `frel` and `fra`, leave the accesses relaxed and turn each program-order edge into a fence edge of that order.

The CPU order is per ISA. AArch64 has five choices. `plain` passes the cycle through as it is, so every access is a plain `STR` or `LDR`. `ra` appends diy's release atom `L` to every write and its AcquirePC atom `Q` to every read, so `PodWW` becomes `PodWWLL` and both stores render as `STLR`, while a read renders as `LDAPR`. These lower to an AArch64 store-release and an RCpc load-acquire, respectively. The load is `LDAPR` rather than the RCsc `LDAR`. `LDAR` would additionally order the load after a preceding `STLR` when the two instructions access different addresses [46]. The annotations under test do not require this ordering. `sy`, `st` and `ld` replace every program-order edge by a barrier edge, so `PodWW` becomes `DMB.SYdWW`, a `DMB SY` between the two writes, or the same with `DMB ST` or `DMB LD`.

On x86_64, the choices are `plain` and `mf`, an `MFENCE` on every program-order edge, and nothing else. Under x86-TSO, ordinary accesses to write-back memory keep every program order except a store followed by a load from a different location, and `MFENCE` restores that one [8]. A release store or an acquire load therefore needs no special instruction on x86 and is an ordinary `MOV`. Under this model, `LFENCE` and `SFENCE` add no distinct architectural ordering, leaving `MFENCE` as the only non-plain CPU order.

CPU-side ordering is varied only when the GPU operations use system scope, the only GPU scope that includes CPU threads (Section 2.3). Variants at CTA or device scope therefore use `plain` CPU accesses.

A test is named `<shape>-<cut>-<scope>-<cpu>.<gpu>`, with `-x86_64` appended in the x86_64 corpus: `MP-cg-cta-plain.acq` has a plain CPU writer and a GPU reader whose reads are CTA-scoped acquires, and `MP-cg-sys-ra.fsc` a CPU writer whose stores are `STLR` against a GPU reader with an `sc` fence between its relaxed reads, both at system scope. Some order choices have no effect on a particular device assignment. For example, applying `acq` to a GPU column containing only writes leaves every access relaxed, producing the same test as `rlx`; such duplicates are removed.

Table 5.1: The shape catalogue: base cycle, processor count, device-cut classes after symmetry reduction, and the tests of the AArch64 heterogeneous corpus each shape yields.

Shape	Base cycle [31, 32, 47]	Procs	Cuts	Tests
MP	`PodWW Rfe PodRR Fre`	2	2	98
SB	`PodWR Fre PodWR Fre`	2	1	63
LB	`PodRW Rfe PodRW Rfe`	2	1	63
2+2W	`PodWW Coe PodWW Coe`	2	1	49
R	`PodWW Coe PodWR Fre`	2	2	112
S	`PodWW Rfe PodRW Coe`	2	2	112
WRC	`Rfe PodRW Rfe PodRR Fre`	3	6	295
RWC	`Rfe PodRR Fre PodWR Fre`	3	6	295
ISA2	`PodWW Rfe PodRW Rfe PodRR Fre`	3	6	350
WWC	`Rfe PodRW Rfe PodRW Coe`	3	6	309
WRW+2W	`Rfe PodRW Coe PodWW Coe`	3	6	281
WRW+WR	`Rfe PodRW Coe PodWR Fre`	3	6	309
WRR+2W	`Rfe PodRR Fre PodWW Coe`	3	6	267
W+RWC	`PodWW Rfe PodRR Fre PodWR Fre`	3	6	350
Z6.0	`PodWW Rfe PodRW Coe PodWR Fre`	3	6	364
Z6.1	`PodWW Coe PodWW Rfe PodRW Coe`	3	6	336
Z6.2	`PodWW Rfe PodRW Rfe PodRW Coe`	3	6	364
Z6.3	`PodWW Coe PodWW Rfe PodRR Fre`	3	6	322
Z6.4	`PodWW Coe PodWR Fre PodWR Fre`	3	6	364
Z6.5	`PodWW Coe PodWW Coe PodWR Fre`	3	6	336
3.2W	`PodWW Coe PodWW Coe PodWW Coe`	3	2	98
3.SB	`PodWR Fre PodWR Fre PodWR Fre`	3	2	126
3.LB	`PodRW Rfe PodRW Rfe PodRW Rfe`	3	2	126
IRIW	`Rfe PodRR Fre Rfe PodRR Fre`	4	8	344
IRWIW	`Rfe PodRW Coe Rfe PodRW Coe`	4	8	366
IRRWIW	`Rfe PodRR Fre Rfe PodRW Coe`	4	14	661
CoRR	`Rfe PosRR Fre`	2	2	49
CoWR	`PosWR Fre Coe`	2	2	57
CoRW2	`Rfe PosRW Coe`	2	2	57
Total			135	6923

5.2 Heterogeneous litmus format

The litmus format is established for CPU ISAs, and a CPU column is reused as it is. A GPU thread has no established column form, and HetLitmus proposes one: every access carries its memory order and scope (Section 2.3) in HetLitmus's common Bell/LISA vocabulary (Subsection 5.1.3), and the file carries a scope hierarchy for the GPU threads. The initial state and the final condition name registers of a thread or memory locations, so they need no change.

What remains is that litmus7 assumes one ISA per test: the header's architecture token selects one lexer, one parser and one compile chain for every processor. A heterogeneous test therefore needs a format that names the device of each processor (Subsection 5.2.1), and a way to carry two ISAs through litmus7 while its existing parsing and compilation code keeps seeing one architecture (Subsection 5.2.2).

5.2.1 Per-thread device tags

The header token is `Het`, a new constructor of `Archs.t`, read by the splitter with no grammar change. Every processor carries a device tag, parsed into an annotation slot `MiscParser.proc` already had. A pre-scan of the raw program text settles one CPU ISA for the whole test. Listing 5.3 is the reference test: the writer on the CPU, the reader on the GPU with acquire reads at system scope.

```
Het MP-cg-sys-plain.acq
"Heterogeneous MP-cg-sys-plain.acq: per-proc device assignment cpu,gpu (cpu=AArch64,
    gpu=LISA)"
{
0:X1=x;
0:X3=y;
}
 P0:cpu      | P1:gpu              ;
 MOV W0,#1   | r[acquire,sys] r0 y ;
 STR W0,[X1] | r[acquire,sys] r1 x ;
 MOV W2,#1   |                     ;
 STR W2,[X3] |                     ;
scopes: (sys (gpu (cta P1)))
exists (1:r0=1 /\ 1:r1=0)
```

Listing 5.3: `MP-cg-sys-plain.acq`

litmus7 checks the test before writing any file, so a refusal exits non-zero and leaves nothing on disk. The checks cover the device tags, which must name at least one GPU processor, the scope tree, the Bell vocabulary of the GPU column and the vendor target.

5.2.2 Two ISAs in litmus7

A test with two ISAs poses two problems: representing both kinds of instruction in one program and parsing both from one file. One functor takes the CPU and GPU backends and builds a compound architecture whose registers, barriers and instructions are tagged as CPU or GPU. Each operation on them is forwarded to the CPU or GPU backend according to that tag.

Each instruction parser accepts only one ISA grammar, so none can parse the mixed program table. HetLitmus instead supplies a custom table parser (Figure 5.2). It reads the program section as text, extracts the `scopes:` row, splits the remaining table at its row and cell separators, and transposes it into one column per processor. The processor's device tag selects the AArch64, x86_64 or LISA/Bell per-column parser. The existing scope grammar reads the extracted tree.

After the columns are parsed, the unchanged `GenParser` treats the compound architecture as one architecture module with one pseudo-instruction type. The Het-specific emitter then separates the two halves. It projects the CPU half back into a single-ISA test and reuses litmus7's register allocator, compiler and assembly-function dumper; a new file emitter wraps those functions in `<t>_cpu.c`. The GPU half goes to the vendor emitter.

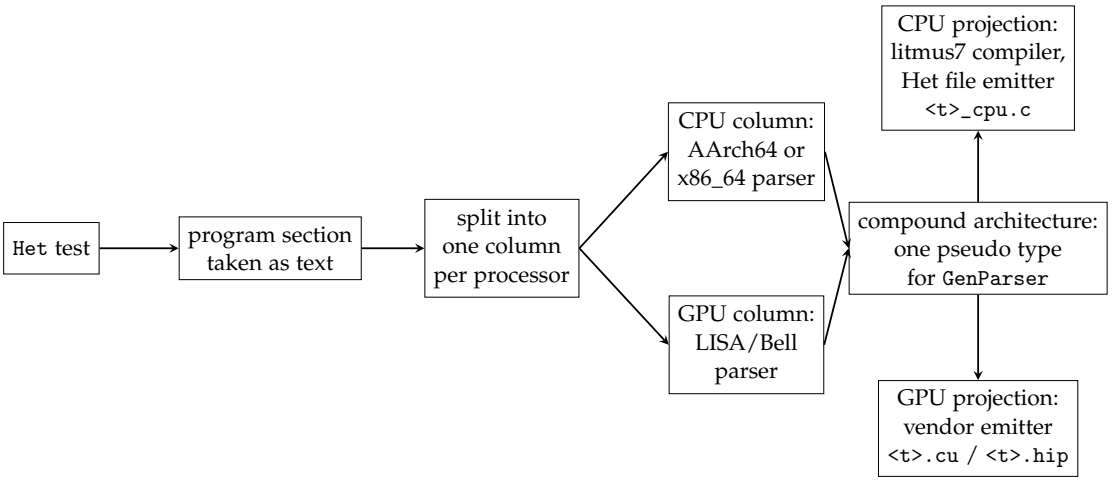


Figure 5.2: One Het test through litmus7.

5.3 Harness emission

litmus7 compiles a CPU test into C with inline assembly for one ISA and runs the processors as POSIX threads of one process [13]. Executing a heterogeneous test additionally requires a GPU kernel in a vendor language, a host driver that allocates the shared memory and launches it, and build files that compile the two halves separately and link them into one binary. Most of a harness is the same whichever vendor it targets, so the emitter writes that part once, and a vendor supplies only its instruction lowering and one record of what else differs.

5.3.1 Harness contents

One `.litmus` becomes one directory named after the test, as shown in Listing 5.4. `int main(void)`, the POSIX-thread wrappers and the GPU kernel are emitted into `<t>.cu` or `<t>.hip`. `<t>_cpu.c` contains the litmus7-generated inline-assembly functions for the tested CPU code. Four runtime headers provide the rendezvous (Subsection 5.4.1), the CPU and GPU stress helpers and host-side interconnect noise (Subsection 5.4.2), and the outcome rule (Subsection 5.5.1). The GPU-side noise loop is emitted in the same vendor source file. The headers are copied into each harness unchanged, together with litmus7's outcome-histogram files. Between the CUDA and HIP directories of one test, only `Makefile`, `comp.sh`, `README.md` and the vendor source file differ. Each difference is a field of the dialect record. The fields include: the compiler and default target (`nvcc sm_90`, `hipcc gfx942`), the runtime include (`<cuda/atomic>`, `<hip/hip_runtime.h>`), the launch call (`cudaLaunchCooperativeKernel`, `hipLaunchCooperativeKernel`), and the allocator of the shared locations (Table 5.5). A harness directory is therefore self-contained and can be copied to the device machine and built there with no herdtools7 checkout.

```
MP-cg-sys-plain.acq.cu     GPU kernel, GPU noise loop and host driver, CUDA dialect
MP-cg-sys-plain.acq_cpu.c  tested CPU body: litmus7's AArch64 inline asm
outs.c  outs.h             litmus7's outcome histogram, verbatim from litmus/libdir
het_verdict.h              record, rule, aggregate
het_cpu_stress.h           CPU stress and host-side interconnect noise
het_stress.h               GPU stress
het_rdv.h                  rendezvous and slot arithmetic
comp.sh  Makefile          compile-only build, plus the link target
README.md                  file roles, build and link commands
```

Listing 5.4: The emitted harness directory of `MP-cg-sys-plain.acq`, CUDA target

5.3.2 Annotation lowering

Table 5.2 gives the constants and fence tokens, and Listing 5.5 the operation forms. Every CUDA access, relaxed ones included, is a `cuda::atomic_ref` operation, while a fence is inline PTX, because `cuda::atomic_thread_fence` lowers acquire, release and acquire-release requests alike to `fence.acq_rel` and therefore cannot preserve a one-sided acquire or release fence [48]. An `sc` access has no PTX load or store form of its own: libcu++ lowers it to the scope's `fence.sc` followed by a relaxed store or an acquire load, so it is two PTX operations [10, 48].

The CPU side is lowered by the generator, litmus7 compiling the literal mnemonics as any test of that ISA (Table 5.3). The CPU file carries that assembly between `#START` and `#END` markers under a host-ISA guard, with an `#error` naming the ISA and the clang triple otherwise. Every emitted AArch64 harness is built with `-march=armv8.3-a`; tests containing `LDAPR` require it because the assembler's default base rejects that instruction. The x86_64 profile uses no additional ISA flag.

Table 5.2: The constant each Bell tag lowers to per vendor [48, 49].

Bell	CUDA	HIP
relaxed	cuda::memory_order_relaxed	__ATOMIC_RELAXED
acquire	cuda::memory_order_acquire	__ATOMIC_ACQUIRE
release	cuda::memory_order_release	__ATOMIC_RELEASE
acqrel	fence acq_rel	fence __ATOMIC_ACQ_REL
sc	cuda::memory_order_seq_cst	__ATOMIC_SEQ_CST
cta	cuda::thread_scope_block	__HIP_MEMORY_SCOPE_WORKGROUP; fence "workgroup"
gpu	cuda::thread_scope_device	__HIP_MEMORY_SCOPE_AGENT; fence "agent"
sys	cuda::thread_scope_system	__HIP_MEMORY_SCOPE_SYSTEM; fence ""

```
CUDA, each access in a braced block of its own
  w[o,s] x v    cuda::atomic_ref<int, S> ref(*slot);  ref.store(v, O);
  r[o,s] r0 x   cuda::atomic_ref<int, S> ref(*slot);  r0 = ref.load(O);
  f[o,s]        asm volatile("fence.<sem>.<s>;" ::: "memory");

HIP
  w[o,s] x v    __hip_atomic_store(slot, v, O, S);
  r[o,s] r0 x   r0 = __hip_atomic_load(slot, O, S);
  f[o,s]        __builtin_amdgcn_fence(O, "<syncscope>");
```

Listing 5.5: The GPU lowering of each Bell operation. `O` (order) and `S` (scope) are the constants of Table 5.2, `sem` the PTX name of the order, and `slot` the location's address.

Table 5.3: The CPU profiles: role or token, representative generated text and instruction [50, 8].

ISA	Role or token	Generated text	Instruction
AArch64	plain write, read	`MOV W0,#1; STR W0,[X1]; LDR W0,[X1]`	store, load
AArch64	release `L`, acquire `Q`	`STLR W0,[X1]; LDAPR W0,[X3]`	store-release, load-acquire
AArch64	`sy`; `st`; `ld`	`DMB SY`; `DMB ST`; `DMB LD`	full, store, load barrier
x86_64	plain write, read	`movl $1,(x); movl (x),%eax`	store, load
x86_64	`mf`	`mfence`	full barrier

5.3.3 Faithfulness checks

Faithfulness is the claim that an emitted harness carries exactly the memory order, scope and operation kind its annotation specifies, with no weakening, strengthening, misplacement or missing qualifier.

Because parsing generated code and matching it to the original litmus test must account for a wide range of instruction forms at different levels of generated code, the checking logic is complex and remains under development.

NVIDIA: on the generated PTX. A harness is compiled with `nvcc --ptx`, and the test's own operations are read back out of the PTX. Each is reduced to a kind, an order and a scope, and that sequence must equal the `.litmus` annotations element by element, an `sc` access counting as its fence and its access. On a heterogeneous test the checker forms one expected stream by concatenating the GPU lanes. Each lane contributes first its rendezvous (Subsection 5.4.1), an arrival and a poll that must be relaxed, system-scoped and fence-free, followed by its annotated operations. The concatenated AArch64 memory and barrier mnemonic stream must equal the one extracted from the inline-assembly literals in the emitted `_cpu.c`. Every system-scope operation in the kernel must belong to the expected stream, so neither the stress layers nor the code around the tested instructions, such as the register recording and the iteration bookkeeping, may carry one.

AMD: on the emitted HIP source. HetLitmus does not perform the analogous check on HIP compiler output. The emitted `__hip_atomic_load`, `__hip_atomic_store` and `__builtin_amdgcn_fence` calls are Clang builtins [51, 52], not inline assembly, and `hipcc` lowers them to generation-specific machine code. The AMDGPU backend specifies, for example, a system-scope release-store sequence with waits between a generation-specific writeback and the store: `buffer_wbl2` on `gfx90a`, `buffer_wbl2 sc0=1 sc1=1` on `gfx942`, and `global_wb scope:SCOPE_SYS` on `gfx12` [11]. A check on

the generated assembly would therefore need an expected sequence per generation, and none is kept. The comparison is therefore made at the source level, and over the whole lane: a test gate rebuilds from the `.litmus` the complete lane the emitter is to write for each GPU column, with every model operation as the builtin and constants of Listing 5.5 and Table 5.2, and compares it with the emitted HIP source line for line, so that nothing may be added, dropped or reordered around the operations either.

5.4 Run environment

litmus7 upstream does not synchronise a kernel lane with a pthread per iteration, nor stress the GPU or the interconnect. The heterogeneous run environment is a new contribution: one cooperative launch per run whose lanes rendezvous with the CPU threads every iteration, plus stress on both devices and on the interconnect.

5.4.1 Barriered persistent loop

The kernel is launched once per run and loops over the iterations together with the CPU threads, rather than being relaunched per iteration, to avoid a kernel launch latency in the microsecond range [53] every iteration. By default, the test executes `NUMBER_OF_RUN` runs of `SIZE_OF_TEST` iterations; the two are litmus7's `-r` and `-s`, with defaults 10 and 100000. There are `NPART` participants, the test's own CPU threads and GPU lanes. Stress blocks, noise blocks, CPU stress threads and host noise threads do not count towards this. Every shared location is an array `SIZE_OF_TEST` slots wide and iteration n uses slot n on both devices; consecutive slots sit 32 `int`s, or 128 bytes, apart. Figure 5.3 illustrates one iteration; a detailed explanation follows below.

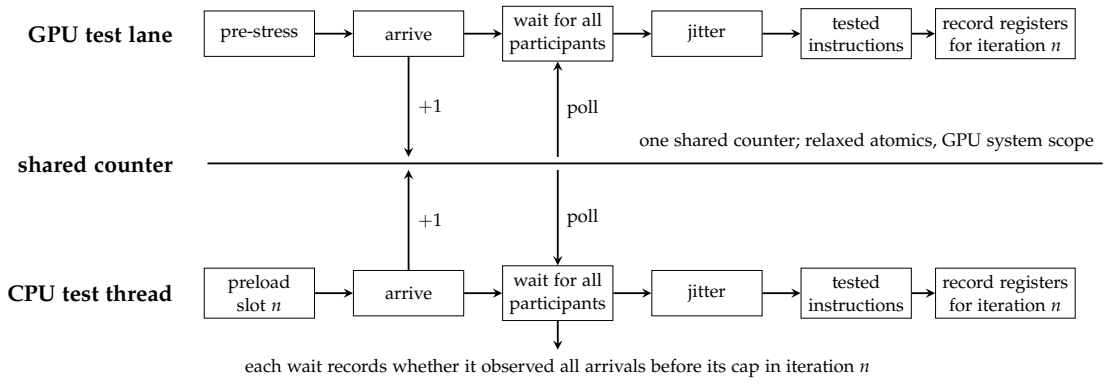


Figure 5.3: One iteration of the barriered persistent loop.

A GPU lane's pre-stress, ported from cuda-litmus [54], accesses a device scratchpad disjoint from the test locations, while a CPU thread's preload reuses litmus7's cache treatments [34] on the shared lines it is about to touch. The pre-stress decision is drawn per lane and iteration, and the preload decision per location and iteration, so a run can mix stressed with unstressed iterations and warm with cold locations.

The rendezvous is a relaxed `fetch_add` of 1 on a shared `uint64_t` counter, at system scope on the GPU, followed by relaxed polling to `NPART*(n+1)`. Iterations count from 0, so each participant has arrived $n+1$ times by the end of iteration n. The wait returns 1 if it observes the target before its cap, and 0 otherwise. Both operations use relaxed memory order. On the Hopper GPU an acquire at `gpu` scope or wider appears to invalidate the local L1 cache [7], so with an acquire poll a stale line the coming iteration could have read would be gone before the tested accesses ran. A weak outcome would therefore be suppressed by the rendezvous rather than by the test annotations.

The poll caps are overridable at run time. Once every `HET_RDV_POKE_EVERY` polls, the host arm invokes `cudaStreamQuery(0)` on CUDA. CUDA does not guarantee device progress while the host merely spins; repeatedly calling a CUDA query API inside the spin provides the guide's forward-progress guarantee [55]. The query is enabled only on the first iteration and after a failed rendezvous. The release jitter is a draw per participant per iteration of 0 to `HET_RELEASE_JITTER_SPINS` empty spins, so successive iterations vary the relative phase of the two devices instead of repeating one alignment.

The tested instructions are then the column's own, every memory address in them offset to this iteration's slot, so a write to x in iteration n writes element n of x's array. What each load returned has to be kept until the condition is evaluated after the run, so each participant copies each observed register into an array of its own at index n (the value `P0`'s `r0` held in iteration n is stored at index n of `r0`'s array), and the host reads these arrays once the run is over.

After the join, the host decodes every iteration. An iteration is scored only if every participant's wait observed the target before its cap; otherwise that iteration is discarded. A scored iteration builds its outcome from the register buffers and location slots, evaluates the compiled weak-behaviour expression, increments `target_count` if it holds and feeds litmus7's histogram.

5.4.2 Stress and noise

The GPU stress is ported from cuda-litmus [54]. Its stress routine loops over rounds, selecting one of four two-access sequences at a location in a GPU-device allocation: `st;st`, `st;ld`, `ld;st` or `ld;ld`, the two-instruction case of Sorensen et al. [44]. Before each launch, a host routine attempts to distribute the stressing blocks across distinct scratchpad regions, round-robin or in chunks. The pattern reaches the kernel as a

runtime argument; on CUDA, this prevents `nvcc` from deleting the stress traffic entirely, although it may merge the source-level `ld;ld` pair into one scratchpad load per round.[1] A `volatile` lookup of the stress location preserved the pair, but was not used because PTX gives volatile operations relaxed system-scope semantics [10], which would add a system-scope operation to the stress layer. `HET_GPU_MEM_STRESS_PCT`, the percentage of iterations in which the stress blocks run, is 100 by default and is decided by one draw shared by all stress blocks in each iteration, following webgpu-litmus [56].

The CPU side reuses litmus7's cache primitives and affinity recipe [34]. Before the tested accesses, the preload may leave each location untreated or flush, read-prefetch or store-prefetch it. Cores are assigned in numerical order, first to test threads, then to host noise threads and finally to CPU stress threads. `HET_CPU_STRESS_THREADS` counts the stress threads. By default, they use every remaining core except two left for the system. The stress threads walk a shuffled, disjoint scratchpad using the two-access sequences of Sorensen et al. [44]. Every draw on either device uses splitmix64 [57], which HetLitmus evaluates statelessly as a function of seed, participant and index rather than of the draws before it, so that the same three inputs give the same value on either side and a run's pseudorandom choices replay from its seed.

HetLitmus also provides interconnect noise intended to load the host–device data path. Inspired by the noise kernels of Fusco et al. [15], both devices stream-read one system-memory buffer first-touched by the CPU. Fusco et al. instead used separate buffers placed in the memory remote from each reader. The buffer is intended to exceed the target's last-level cache (LLC). The host gives each noise thread one contiguous slice. With the default stride of one, each host thread reads its slice sequentially, while N device threads read words k, $k+N$, $k+2N$, and so on. At each step the GPU threads together read N consecutive words, and that window moves through the buffer and wraps at its end. On CUDA the buffer is a CPU-first-touched `malloc` allocation. On HIP it is `hipMallocManaged`, the fine-grained allocation that stays coherent between the devices while a kernel runs [19]. A refused or failed allocation disables both halves for the run, and a non-sighting run that requested either is then disqualified (Subsection 5.5.1). On an accelerated processing unit (APU) such as the MI300A the two devices share one physical memory [24], so no placement of the buffer can make one device's reads cross into the other's memory. The noise may load the shared path to memory instead. Whether interconnect noise makes weak behaviour more likely has not been measured on either target.

[1] This occurred when compiling the programs used in the evaluation with `nvcc` 12.9.86 for `sm_121`.

5.4.3 Tuning knobs

HetLitmus's tuning method follows the random-search approach of Kirkham et al. [45] and adapts cuda-litmus's joint sampling of a complete stress configuration [54]. Each knob is either a compile-time macro that can be overridden with `-D`, or an environment variable read at run time. Table 5.4 collects the important knobs. `HET_SEED` pins the base seed when set; otherwise the base is 31 random bits from `getrandom`, and the per-run seed is the base plus the run index.

The defaults provide an initial configuration and are not assumed to be optimal for a target. The fixed GPU-stress values and the base `BLOCK_DIM` of 128 reproduce cuda-litmus's committed configuration [54]. `GPU_MEM_STRESS_PCT=100` follows webgpu-litmus's all-stress setting [56]. `NOISE_MB=8192` matches the 8 GB noise buffer of Fusco et al. [15]. The remaining CPU-stress, noise and rendezvous defaults have neither been tuned using hardware measurements nor adopted from prior work. The 50% discard threshold rejects a non-sighting once most iterations failed the rendezvous. Automatic values are derived at emission or launch: `BLOCK_DIM` is the larger of 128 and the required per-block lane count, and `GPU_STRESS_BLOCKS` fills the co-resident capacity left after test and noise blocks.

The tuner `hetlitmus/tune_stress.py` searches that space on the target machine itself. Its input is a short list of tuning tests, which should fire neither too often nor too rarely under the default stress, since either extreme may yield too little variation in observed rates to distinguish configurations. It reads what bounds a draw, the core count and the available memory from the machine, the last-level cache size from a flag and the block width from the emitted harnesses, and then draws configurations at random, one value per knob from that knob's range, redrawing one the machine cannot honour. Each configuration is built once and then runs every tuning test once. A tuning test's score under a configuration is the rate of weak outcomes observed per second.

Winners are chosen per tuning test: the configuration with the highest rate on a tuning test wins it, and the leader is the configuration that wins the most tuning tests. Every configuration that wins at least one tuning test is written out to a parameter file that is passed to the compiler when the harnesses are rebuilt. Parameters tuned on one machine do not necessarily transfer to another [45].

5.4.4 Allocation

The shared locations and the rendezvous counter must lie in memory both devices access coherently, at cache-line granularity, while the kernel runs. Memory that is copied or migrated between the devices makes the test measure the copy, while an

Table 5.4: Selected run-environment knobs, with HET_ omitted. A dagger marks a tuner draw; the tuner also draws GPU_MEM_STRESS_PCT and CPU_STRESS_THREADS described above.

Knob	Default	Effect
Rendezvous and discards		
CAP_GPU	16384	polls a GPU lane makes before giving up on an iteration
CAP_CPU	524288	the same for a CPU thread
RELEASE_JITTER_SPINS	64	upper bound of the release delay, in empty spins
RDV_MAX_DISCARD_PCT	50	discard percentage above which a non-sighting is disqualified
GPU stress		
GPU_SPREAD†	9	requested number of distinct scratchpad regions
GPU_WORDS_PER_REGION†	16	number of 4-byte words per logical region
GPU_STRESS_ASSIGN†	chunking	how blocks map onto the regions: round-robin, or contiguous chunks
GPU_MEM_STRESS_ROUNDS†	445	maximum stress rounds executed by each lane per call
GPU_MEM_STRESS_PATTERN†	0 (st;st)	which of the four two-access sequences the stress blocks run
GPU_PRE_STRESS_PCT†	65	percentage of iterations in which a test lane stresses itself first
GPU_PRE_STRESS_ROUNDS†	57	maximum rounds in one such self-stress
GPU_PRE_STRESS_PATTERN†	3 (ld;ld)	the two-access sequence of the self-stress
CPU stress		
CPU_SCRATCH_WORDS†	262144	size of the stress scratchpad in 8-byte words
CPU_SPREAD†	8	distinct regions a stress thread hits
CPU_WORDS_PER_REGION†	8	number of 8-byte words per logical region
CPU_STRESS_PATTERN†	0 (st;st)	the two-access sequence of the stress threads
CPU_PRELOAD_PCT†	50	chance of applying a cache treatment to a location; redrawn for every location and iteration
How much of the machine		
BLOCK_DIM†	128	number of threads in one block
GPU_STRESS_BLOCKS†	auto	stress blocks; auto takes every block the co-resident grid has left after the test and noise blocks
GPU_NOISE_BLOCKS†	8	blocks streaming the noise buffer
CPU_NOISE_THREADS†	1	host threads streaming it
NOISE_MB†	8192	noise-buffer size, intended to exceed the LLC
NOISE_STRIDE†	1	words between consecutive noise reads
CPU_RESERVE_CORES	2	cores left unpinned for the system
Campaign, read at run time		
RUNS_MAX	-r	runs this invocation makes, at most the compiled count
STOP_AT_SIGHTING	0	1 ends the run loop at the first clean sighting
SEED	from entropy	base seed for the per-run pseudorandom choices

allocation that one device may not access while the other's program runs cannot support the test and is rejected. Which allocation gives that depends on the platform, so the allocator is a run-time parameter, `HET_ALLOC`. The allocation classes are introduced in Section 2.1; Table 5.5 gives their vendor mappings and the modes HetLitmus rejects. Memory only the GPU touches is plain `cudaMalloc` or `hipMalloc`.

`auto` selects the default allocation for the chosen backend. On CUDA, it uses system `malloc` when `pageableMemoryAccess` is 1 and `cudaMallocManaged` otherwise. When the attribute is 0, system allocations are not necessarily accessible to the GPU, so `cudaMallocManaged` provides the required explicit managed allocation [17]. On HIP, `auto` always uses `hipMallocManaged`. This provides fine-grained memory that supports coherent CPU–GPU operations during kernel execution [18]. Before allocating, the backend checks `managedMemory` and `concurrentManagedAccess`; if either is 0, it exits with status 2. `HET_ALLOC` can instead request `malloc`, `managed` or `pinned` explicitly on CUDA, whereas HIP accepts only `managed`. On CUDA, managed allocation also requires `concurrentManagedAccess`, which permits the CPU to access the allocation while the kernel runs [20]. The `pinned` mode uses mapped `cudaHostAlloc`, which provides page-locked host memory directly accessible from kernel code. `hostNativeAtomicSupported` indicates whether the rendezvous read–modify–write is natively atomic [17].

Table 5.5: Allocation modes of the shared locations, per vendor.

`HET_ALLOC`	CUDA	HIP
unset, `auto`	`malloc` with pageable memory access, else `managed`	fine-grained `hipMallocManaged`; exit 2 without managed-memory or concurrent-managed-access support
`malloc`	system `malloc`; exit 2 without pageable access	exit 2
`managed`	`cudaMallocManaged`; exit 2 without concurrent managed access	`hipMallocManaged`, the one mode; same guards as `auto`
`pinned`	`cudaHostAlloc` mapped and portable; a warning without host-native atomics	exit 2
other	exit 2	exit 2

5.5 Campaign and reporting

This section covers what a run reports, the per-run outcome and its caveats (Subsection 5.5.1), and the stop rule that ends a test across runs (Subsection 5.5.2).

5.5.1 Run outcomes and caveats

A run's outcome is OBSERVED, NOT-OBSERVED or COLD-INVALID. If the weak outcome was seen in at least one scored iteration, the run is OBSERVED. Otherwise the run's record is checked for disqualifiers, such as a rendezvous that never worked or a stress mechanism that was requested but never ran. Any disqualifier makes the run COLD-INVALID; without one it is NOT-OBSERVED. COLD-INVALID therefore identifies problems in the run environment that may be corrected or retuned before another run. A second class of flags, caveats, is reported with the outcome without changing it, such as a failed core pin or rendezvous caps never calibrated on the target. Table 5.6 lists the disqualifiers and the caveats.

Each run prints a rendezvous line, the HetObs line and the verdict block, and after the last run the invocation prints the HetStats line, a human summary and litmus7's histogram (Listing 5.6).

Table 5.6: Flags of the outcome rule, prefixes HET_DQ_, HET_CV_ and HET_REQ_ dropped.

Class	Flag	Meaning
disqualifier	RDV_DEAD	decoding pass never ran, every iteration was discarded, or more than the allowed share was
	STRESS_TRUNCATED	the stress stopped while the tested lanes still ran
	GPU_STRESS_DEAD	GPU stress requested, but no round ran
	CPU_STRESS_DEAD	stress threads requested, but no round ran
	CPU_PRELOAD_DEAD	preload requested, but no cache operation was issued
	CPU_NOISE_DEAD	host noise requested, but no round ran
	GPU_NOISE_DEAD	device noise requested, but no block ran
caveat	UNSTRESSED	no stress was requested at all
	ONE_OUTCOME	every scored iteration read back the same outcome
	RDV_UNCALIBRATED	the rendezvous ran with placeholder caps
	AFF_FAILED	a core pin failed
request	GPU_STRESS, CPU_STRESS, CPU_PRELOAD, CPU_NOISE, GPU_NOISE	the mechanisms this run asked for; a mechanism is dead only if requested here and idle

```
HetLitmus rendezvous: scored=S discarded=D (cap_cpu=c cap_gpu=g) caps=CAPC/CAPG jitter=J
     discard_max=P% caps UNCALIBRATED
HetObs T run=n N=N scored=S discarded=D target=K cap_cpu=c/CAPC cap_gpu=g/CAPG
    calibrated=0 vary=v stress_trunc=t do_stress_rounds=r req=0xREQ stress_threads=e
    stress_rounds=er stress_acc=ea preload=p cpu_noise=nr/nww gpu_noise=nb/ng noise_ws=
    wMB aff_fail=a
HetVerdict T run=n: OBSERVED
  ** T: the weak outcome was OBSERVED on (AArch64, cuda) in K of the S scored iteration(
     s) of N=N (P%).
```

```
  <one sentence per set caveat bit>
HetStats T obs=CLASS R=R usable=u k=k k_eff=ke degen=d N=N scored=S discarded=D
HetStats T: R run(s), u usable, observed in k (ke after the decode guard).
  OBSERVED in k of R run(s), ke of them after the decode guard.
Test T
num0 > 1:r0=1; 1:r1=1;
num1 *> 1:r0=1; 1:r1=0;
```

Listing 5.6: The per-run printout of an observed run, the statistics lines and litmus7's histogram, fields abbreviated; the asterisk marks the weak outcome.

5.5.2 Stop rule

`hetlitmus/campaign.py` re-invokes a test's binary with fresh seeds, each invocation under a timeout of 900 s by default, adding up the `HetStats` counters across invocations until the stop rule assigns the test a terminal state over these totals. A test ends `OBSERVED` at the first run that shows the weak outcome without the `ONE_OUTCOME` caveat; a readout that never varies may indicate a stale buffer rather than a behaviour of the hardware [37]. Under `HET_STOP_AT_SIGHTING` the harness stops there itself. Without a clean sighting the test ends `BUDGET` when its runs, 100 by default, are exhausted. Under `--rate` a sighting stops nothing and every test runs to its budget, so that a test that fires yields a rate. A test ends `ERROR`, without retry, when its harness is missing, times out, exits non-zero, prints no parsable `HetStats` line, adds no runs, or has no usable run at all. The state CSV, rewritten after every test, holds each test's terminal state, its number of successful invocations, and its pooled counters, together with the campaign's seed base, drawn once and replayable through `--seed0`.

6 Evaluation

This chapter evaluates HetLitmus on a DGX Spark. Section 6.1 describes the platform and the evaluation set, Section 6.2 how the campaign ran, and Section 6.3 what it observed.

6.1 Experimental setup

6.1.1 Platform

The DGX Spark was rented from Vast.ai, a GPU cloud marketplace [58]. It ran Linux kernel `6.17.0-1029-nvidia`, NVIDIA driver 580.173.02 and CUDA 12.9, with `nvcc` 12.9.86. The probe (Section 4.3) reported compute capability 12.1, 48 SMs and a 24 MiB L2 cache, so every harness was compiled for `sm_121`. It also reported `pageableMemoryAccess=1`, so the default allocation mode placed the shared locations in system `malloc` memory (Subsection 5.4.4). With `hostNativeAtomicSupported=1`, the rendezvous `fetch_add` is natively atomic between the devices. Generation, emission and build used herdtools7 branch `hetlitmus-work` at commit `64705a6`. The logs and results of every run are available at `https://github.com/lenguyenhcm325/hetlitmus-gb10-run-data`.

The GH200 was planned as a second machine. Free credits were obtained from the cloud provider Vultr, which offers the GH200 as a cloud instance [59], but while the evaluation was planned and carried out, no on-demand instance was available there, only reserved instances with a term of at least twelve months, so no evaluation on it was possible.

6.1.2 Evaluation set

The AArch64 heterogeneous corpus holds 6,923 tests (Table 5.1), and running all of them would take a long time. The evaluation therefore runs a subset of 637 tests, selected by `hetlitmus/tests/subset.py` with three rules. The unordered part, 135 tests, runs every shape under every device cut with nothing ordered. The fully fenced part, another 135, runs them with every program-order edge fenced on both devices. The full grid adds the remaining 367 tests, of six shapes chosen for the access pairs they cover. Between them, MP, SB and LB put every two-location access pair, write–write,

read–read, write–read and read–write, on each device. CoRR, CoWR and CoRW2 put the read–read, write–read and read–write pairs on one location. MP and SB are introduced in Section 2.2; Figure 6.1 gives the other four.

```
LB                                  CoRR
 P0          | P1          ;         P0          | P1          ;
 r0 = x      | r1 = y      ;         r0 = x      | x = 1       ;
 y = 1       | x = 1       ;         r1 = x      |             ;
exists (r0=1 /\ r1=1)               exists (r0=1 /\ r1=0)

CoWR                                CoRW2
 P0          | P1          ;         P0          | P1          ;
 x = 2       | x = 1       ;         r0 = x      | x = 2       ;
 r0 = x      |             ;         x = 1       |             ;
exists (r0=0 /\ x=2)                exists (r0=2 /\ x=2)
```

Figure 6.1: The test shapes of LB, CoRR, CoWR and CoRW2 [32].

6.2 Campaign

6.2.1 Stress tuning

Before the campaign, the tuner (Subsection 5.4.3) searched the stress knobs on the DGX Spark. Its tuning tests, on which it measured the rate of weak behaviour, were chosen by three criteria:

1. each test had shown its weak outcome in an earlier run on a DGX Spark, since a test that no configuration reveals cannot rank any;
2. each fired neither too often nor too rarely under the default stress, because either extreme may leave too little variation in observed rates for useful comparison across configurations;
3. together they cover a broad range of tests: shapes of two, three and four threads, all three scopes, and fenced as well as unfenced tests.

These criteria selected 12 tuning tests. The tuner drew 50 configurations and ran each once on every tuning test.

Interconnect noise (Subsection 5.4.2) was off in the search and in the campaign. Memory stress is an established means of exposing weak behaviour on GPUs. The interconnect noise kernels HetLitmus adapts, in contrast, were used to measure bandwidth under load [15], and their effect on weak outcomes has not been measured in

prior work. Noise and stress also share one fixed pool of resources, the CPU cores and the co-resident GPU blocks (Subsection 5.4.2, Subsection 5.4.3). Every core or block given to noise could instead be assigned to stress, so enabling noise gives up potential capacity for a mechanism with an established effect in exchange for one whose effect is unknown.

6.2.2 Passes

The campaign ran in passes, each under one winning configuration, with the harnesses rebuilt from that configuration's parameter file. The 50 configurations produced 10 winners for the 12 tuning tests. The first pass ran the leader (Subsection 5.4.3), configuration 5, which won 3 tuning tests, on all 637 tests. Every later pass ran one winner only on the tests that no earlier pass had observed. A test observed in one pass is not run again, so each pass runs at most as many tests as the one before. Each run has 100,000 iterations; Table 6.1 gives each pass's budget of runs per test. A test observed in no pass was thus run 150 times, 15 million iterations under 10 configurations. Most observations came from the first pass: with 4,791 of the 67,347 runs, it observed 179 of the 193 observed tests, while the 10 later passes, with the other 62,556 runs, observed only 14 more.

Table 6.1: The campaign passes in run order.

Pass	Configuration	Budget	Tests	Observed	Runs
1	5	10	637	179	4,791
2	39	10	458	5	4,549
3	13	10	453	0	4,530
4	5	50	453	8	22,356
5	43	10	445	0	4,450
6	47	10	445	0	4,450
7	18	10	445	0	4,450
8	46	10	445	0	4,450
9	30	10	445	1	4,441
10	10	10	444	0	4,440
11	4	10	444	0	4,440
Total				193	67,347

The 14 tests observed after the first pass each showed only 1 to 3 weak outcomes in 100,000 iterations in the run that saw it, 11 of them exactly 1, against a median of 851 for the 179 tests the first pass observed. The fourth pass reran the leader under new seeds with a higher budget and observed 8 of them, 4 within its first 10 runs, the budget of every other pass. Of the remaining 6, configuration 39 observed 5 and

configuration 30 observed 1, while the other 7 configurations observed none.

6.2.3 Run validity

Every harness built in every pass, every invocation exited with status 0, and no test ended `ERROR`. No run was `COLD-INVALID` (Subsection 5.5.1), so no requested stress mechanism stayed idle or stopped early, and no run reported a failed core pin. The rendezvous discarded 370 iterations against 6.73×10^9 scored ones, and no run carried the `ONE_OUTCOME` caveat. One configuration ran weaker than drawn: configuration 39 had too few stress blocks to cover the 14 regions requested by `HET_GPU_SPREAD`, so all its stress blocks stressed the same location in the second pass.

6.3 Results

6.3.1 Overview

Of the 637 tests, 193 were observed and 444 were not (Table 6.2). In the run that observed it, a test counted between 1 and 13,579 weak outcomes in 100,000 iterations, with a median of 761; 38 tests counted fewer than 10. The 14 largest counts, from 7,504 to 13,579, were all in SB.

No test in the fully fenced part was observed. All 103 tests in the unordered part with program-order edges on both the CPU and the GPU were observed. Of the 16 with program-order edges on the CPU only, 5 were observed, and of the 16 with them on the GPU only, 1 was, each with at most 58 weak outcomes in 100,000 iterations.

In the unordered part, the count also falls with the number of threads, from a median of 3,168 for two threads to 22 for four. A weak outcome over more threads might be rarer because more threads must interleave in a particular order within the same iteration. For three-thread shapes, the median is 1,043 for tests with one GPU thread and 171 for those with two.

None of the 163 coherence tests, CoRR, CoWR and CoRW2, was observed. On the GH200, which also joins its CPU and GPU over NVLink-C2C, Bagchi et al. [7] infer that the CPU's last-level caches and the GPU's L2 cache are kept coherent by a single-writer protocol; if the GB10 does the same, this might explain why no coherence test showed weak behaviour.

6.3.2 The full grid

Table 6.3 shows the full grid of MP, SB and LB by scope and by the order on each side; CoRR, CoWR and CoRW2 are left out, since none of their tests was observed.

Table 6.2: Observed tests per part of the evaluation set and within the unordered part. Median: weak outcomes in 100,000 iterations in the run that observed the test, over the observed tests.

	Tests	Observed	Median
Part			
Unordered	135	109	303
Fully fenced	135	0	–
Full grid	367	84	1,238.5
Unordered part, program-order edges on			
both devices	103	103	371
CPU threads only	16	5	3
GPU threads only	16	1	4
Unordered part, threads			
2	15	9	3,168
3	90	80	375
4	30	20	22
Unordered part, three threads			
1 on the GPU	45	41	1,043
2 on the GPU	45	39	171

In the `rlx` column, where the GPU accesses are relaxed, every test was observed, including those in the `sy` rows, with a full barrier on every CPU program-order edge. A CPU barrier alone therefore did not remove the weak outcome.

At CTA scope, `acq`, `ra`, `sc` and `facq` on the GPU side of MP-cg and LB-cg were never observed, while `fsc`, `frel` and `fra` were, in MP-cg 2,835, 3,613 and 2,567 times in 100,000 iterations. So at this scope an acquire was never observed, while the sequentially consistent fence `fsc` was. This does not show that the acquire prevents the weak outcome. Each of these tests ran 15 million iterations, and more iterations might still show it.

In MP-gc, where the GPU writes and the CPU reads, every GPU order was observed at CTA and device scope, although at device scope `rel`, `sc`, `fsc`, `frel` and `fra` only 1 to 3 times in 100,000 iterations. At system scope, the only scope that includes CPU threads (Table 2.2), none of the five was observed.

In SB-cg, every order at CTA and device scope was also observed. At system scope, `sc`, `fsc`, `frel` and `fra` were not observed, while `rel` and `ra` were, 1 to 9 times in 100,000 iterations.

SB-cg was observed in 43 of its 63 tests, against 24 of 49 in MP-gc, 11 of 63 in LB-cg and 10 of 49 in MP-cg. Most of the difference lies in `acq` and `ra`, which were observed

Table 6.3: The full grid of MP, SB and LB. • observed, · not observed; a cell in parentheses has no test of its own and shows the result of an identical test: for example, in MP-cg the GPU only reads, so `rel` has no write to apply to and is `rlx`.

		GPU order								
GPU scope	CPU order	`rlx`	`acq`	`rel`	`ra`	`sc`	`fsc`	`facq`	`frel`	`fra`
MP-cg: CPU writes, GPU reads										
`cta`	`plain`	•	·	(•)	(·)	·	•	·	•	•
`gpu`	`plain`	•	·	(•)	(·)	·	·	·	·	·
`sys`	`plain`	•	·	(•)	(·)	·	·	·	·	·
`sys`	`ra`	•	·	(•)	(·)	·	·	·	·	·
`sys`	`st`	•	·	(•)	(·)	·	·	·	·	·
`sys`	`ld`	•	·	(•)	(·)	·	·	·	·	·
`sys`	`sy`	•	·	(•)	(·)	·	·	·	·	·
MP-gc: GPU writes, CPU reads										
`cta`	`plain`	•	(•)	•	(•)	•	•	•	•	•
`gpu`	`plain`	•	(•)	•	(•)	•	•	•	•	•
`sys`	`plain`	•	(•)	·	(·)	·	·	•	·	·
`sys`	`ra`	•	(•)	·	(·)	·	·	•	·	·
`sys`	`st`	•	(•)	·	(·)	·	·	•	·	·
`sys`	`ld`	•	(•)	·	(·)	·	·	•	·	·
`sys`	`sy`	•	(•)	·	(·)	·	·	•	·	·
SB-cg										
`cta`	`plain`	•	•	•	•	•	•	•	•	•
`gpu`	`plain`	•	•	•	•	•	•	•	•	•
`sys`	`plain`	•	•	•	•	·	·	•	·	·
`sys`	`ra`	•	•	•	•	·	·	•	·	·
`sys`	`st`	•	•	•	•	·	·	•	·	·
`sys`	`ld`	•	•	•	•	·	·	•	·	·
`sys`	`sy`	•	•	•	•	·	·	•	·	·
LB-cg										
`cta`	`plain`	•	·	•	·	·	•	·	•	•
`gpu`	`plain`	•	·	·	·	·	·	·	·	·
`sys`	`plain`	•	·	·	·	·	·	·	·	·
`sys`	`ra`	•	·	·	·	·	·	·	·	·
`sys`	`st`	•	·	·	·	·	·	·	·	·
`sys`	`ld`	•	·	·	·	·	·	·	·	·
`sys`	`sy`	•	·	·	·	·	·	·	·	·

in SB-cg at every scope and never in MP-cg or LB-cg. In SB, the GPU thread writes and then reads. An acquire orders only the accesses after a read and a release only those before a write, so neither orders this pair. In the PTX model, only a `fence.sc` between them does [26]. At system scope, `frel` and `fra` were not observed either, although, like `rel` and `ra`, these release and acquire–release fences do not order a write before a later read.

With the GPU side relaxed, SB-cg also counted the most weak outcomes of the four shapes in every row. This might be because in SB a slow write is enough, since the weak outcome needs only that each thread reads before the other thread's write has reached it. In MP and LB, in contrast, a slow write alone is not enough. In MP, the two writes must reach the reader in the opposite order, or the two reads must be performed in the opposite order, and in LB, where each thread reads and then writes, a thread's write must reach the other thread before its own read has returned.

7 Conclusion

This thesis developed HetLitmus, an extension of herdtools7, to test cross-device behaviour on hardware-coherent CPU–GPU systems. On these systems, threads on both devices can access shared memory concurrently, but each device's memory model covers only its own threads. The compound memory model of Goens et al. [12] proposes a framework for composing the device models, but its authors did not evaluate it on physical hardware. This motivated the development of HetLitmus to obtain empirical results on hardware.

To address the gaps identified in Chapter §3, HetLitmus provides one end-to-end workflow for heterogeneous litmus testing. It generates the tests from relaxation cycles: `hetgen7` runs the diy engine once per device and merges the two results into one test. The test uses a new `.litmus` format with one ISA per column, a device tag on every thread and a GPU scope tree, which `litmus7` reads and compiles. Enumerated over shape, device cut, scope and memory order, the generated corpora contain 10,979 tests across 29 shapes. The emitter writes each test as a CUDA or a HIP harness, with one arm per vendor behind a common interface, so a third vendor is supported by adding an arm rather than by redesigning the emitter. Finally, a probe, a build script and a campaign take the harnesses to a results table that reports every observation together with its run-validity information.

The evaluation on a DGX Spark (Chapter §6) shows that this workflow runs successfully end to end on hardware: every harness of the 637-test evaluation set built and ran without errors, and every run was valid. Of the 637 tests, 193 were observed. In the full grid of MP, SB and LB, the weak outcome was observed under every CPU order, including a full barrier, when the GPU accesses were relaxed. With the GPU as the writer and the CPU as the reader in MP, it was observed under every GPU order at CTA and device scope, but under none of the release or sequentially consistent orders at system scope, the only scope that includes CPU threads.

Of the design goals (Section 4.1), systematic coverage and per-device semantics are met by construction: every test is one cell of an enumerated grid, and every column is generated and lowered by the tools of its own architecture. Vendor independence is met at the source level: every Bell operation has a lowering in both arms (Subsection 5.3.2). Faithfulness is checked on the generated PTX and on the emitted HIP source, but the checker is still under development. To raise the chance of observation, the run

environment synchronises the devices in every iteration and places the shared locations in coherent memory. It stresses both devices and tunes the stress on the target with techniques adapted from prior CPU and GPU litmus testing work. Its interconnect noise is implemented, but its effect was not measured.

Two lessons follow from the development. First, the code around the tested instructions can suppress a weak outcome by itself: on the Hopper GPU, an acquire at `gpu` scope or wider appears to invalidate the local L1 cache [7], so an acquire poll in the rendezvous could remove a stale value before the tested accesses run. The rendezvous therefore uses only relaxed operations, and the faithfulness check covers it and the code around the tested instructions as well. Second, a run that observed nothing has to be separated from a run whose environment failed, such as one whose requested stress never ran. A failed environment can then be corrected or retuned before the next run, instead of its non-observation being accepted as a result with no improvement being made.

The evaluation is limited by the hardware accessible during this thesis. Its results come from the DGX Spark, an NVIDIA machine with an AArch64 CPU. The x86_64 corpus and the HIP arm have not run on hardware, and the campaign ran a selected subset of 637 of the 6,923 AArch64 tests. A non-observation states only that the weak outcome was not seen in a limited number of runs. Chapter §8 describes the work that remains to be done.

8 Future Work

Future work includes the first runs on AMD hardware, a third vendor arm for the emitter, and a positive control that would give a non-observation its weight.

8.1 Runs on AMD hardware

No AMD hardware with coherent CPU–GPU memory was accessible during this thesis, so the HIP arm is compile-verified only. The intended targets are the AMD Instinct MI300A and the Ryzen AI Max+ 395.

A first run has to check three things. A probe has to show every participant arriving in every iteration of the rendezvous before any test is read. The shared locations must remain coherent between the devices while the kernel runs. HIP calls this fine-grained coherence (Subsection 5.4.4), but its availability must be verified on the target rather than assumed for every AMD system [19]. The stress parameters (Subsection 5.4.3) must also be tuned again there. Optionally, the generated AMDGPU code should also be inspected on each target, since MI300A and Ryzen AI Max+ use different GPU architectures and may lower the same HIP operations differently.

8.2 Third vendor arm

Intel GPUs require a different backend because they do not natively support CUDA or HIP [60]. An Intel arm would therefore extend the same generated test families to a third GPU architecture and software stack.

The emitter represents CUDA and HIP as sibling entries behind one vendor interface (Section 5.3). Adding a third arm would not require a redesign, but the following extensions:

- a lowering module beside `CudaLang.ml` and `HipLang.ml` that maps each memory order and scope and emits loads, stores and fences.
- a dialect record in `hetDialect.ml` containing the vendor-specific compiler settings, runtime operations and lowering entry point (Subsection 5.3.1). Adding it to the registry makes `litmus7`'s `-gpu-target` option accept the new name.
- vendor-specific allocator fragments for the shared locations and noise buffers.

- third-vendor support in the shared runtime and emission, build and tuning scripts.
- a hardware probe for the target.

8.3 Positive control

A test that ends `NOT-OBSERVED` says nothing about whether this environment could have exposed weak behaviour at all. A positive control closes that gap: a second test, run beside or ahead of the first in the same environment under the same stress, chosen weak enough that it should be observed. As long as it is, the tests beside it ran in an environment that demonstrably exposes weak behaviour.

On x86, for example, SB and R are the shapes to co-run: store buffering is the relaxation x86 exposes most readily, so both should first be established empirically on the target and shared allocation. If they are not, the allocation, insufficient stress and execution conditions should be checked before drawing conclusions about the hardware. The allocation matters because x86-TSO describes write-back memory [8], and on memory mapped uncacheable no reordering is allowed at all [61], so store buffering cannot show there, whatever the CPU. A control closer to each test comes from the mutants of MC Mutants (Section 3.3), run beside the test itself: a twin that fires beside a silent test shows that the annotations under test, not the environment, likely made the difference.

Abbreviations

APU accelerated processing unit

CPU central processing unit

CSV comma-separated values

CTA cooperative thread array

CUDA Compute Unified Device Architecture

CXL Compute Express Link

FPGA field-programmable gate array

GPU graphics processing unit

HBM high-bandwidth memory

HIP Heterogeneous-computing Interface for Portability

IRIW independent reads of independent writes

ISA instruction set architecture

LB load buffering

LISA Litmus Instruction Set Architecture

LLC last-level cache

LOST-POP Local Orderings by Stalling on Threads – Partial-Order Propagation

MP message passing

NUMA non-uniform memory access

PTX Parallel Thread Execution

RWC read-to-write causality

SB store buffering

SC sequential consistency

SM streaming multiprocessor

TSO total store order

WRC write-to-read causality

List of Figures

2.1 Memory organisations of GH200, GB10 and MI300A 5

4.1 The HetLitmus pipeline 18

5.1 Two engine runs, erased to strings and merged by device assignment. . 21
5.2 One Het test through litmus7 26
5.3 One iteration of the barriered persistent loop. 30

6.1 The test shapes of LB, CoRR, CoWR and CoRW2 39

List of Tables

2.1 The MP outcome under SC, x86-TSO and AArch64. 7
2.2 The GPU scope levels common to PTX and AMD. 8
2.3 The GPU memory orders and their effects. 9

5.1 The shape catalogue 24
5.2 The GPU order and scope constants 28
5.3 The CPU profiles: role or token, representative generated text and instruction [50, 8]. 29
5.4 Selected run-environment knobs 34
5.5 Allocation modes of the shared locations, per vendor. 35
5.6 Flags of the outcome rule 36

6.1 The campaign passes 40
6.2 Observed tests per part of the evaluation set and within the unordered part 42
6.3 The full grid of MP, SB and LB 43

Bibliography

[1] A. Smith, G. H. Loh, M. J. Schulte, M. Ignatowski, S. Naffziger, M. Mantor, M. Fowler, N. Kalyanasundharam, V. Alla, N. Malaya, J. L. Greathouse, E. Chapman, and R. Swaminathan. "Realizing the AMD Exascale Heterogeneous Processor Vision: Industry Product." In: *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024, pp. 876–889. DOI: 10.1109/ISCA59077.2024.00068.

[2] J. Evans, M. Andersch, V. Sethi, G. Brito, and V. Mehta. *NVIDIA Grace Hopper Superchip Architecture In-Depth*. NVIDIA Technical Blog. Nov. 2022. URL: https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/ (visited on 09/06/2026).

[3] NVIDIA Corporation. *NVIDIA DGX Spark*. 2026. URL: https://www.nvidia.com/en-us/products/workstations/dgx-spark/ (visited on 09/06/2026).

[4] Y. Yu, N. Oswald, and A. Khandelwal. "CORD: Low-Latency, Bandwidth-Efficient and Scalable Release Consistency via Directory Ordering." In: *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. 2025, pp. 1311–1326. DOI: 10.1145/3695053.3731074.

[5] TOP500. *TOP500 List, June 2026*. 67th edition. June 2026. URL: https://top500.org/lists/top500/2026/06/ (visited on 09/06/2026).

[6] Advanced Micro Devices. *Introducing AMD CDNA 2 Architecture*. 2021. URL: https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna2-white-paper.pdf (visited on 09/24/2026).

[7] S. Bagchi, S. Srivastava, R. Levine, T. Sorensen, R. Stutsman, and V. Nagarajan. "Consistency and Coherence of the NVIDIA Grace-Hopper Superchip." In: *Proceedings of the 2026 ACM SIGPLAN International Symposium on Memory Management (ISMM '26)*. 2026, pp. 66–79. DOI: 10.1145/3814942.3816134.

[8] P. Sewell, S. Sarkar, S. Owens, F. Zappa Nardelli, and M. O. Myreen. "x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors." In: *Communications of the ACM* 53.7 (2010), pp. 89–97. DOI: 10.1145/1785414.1785443.

[9] J. Alglave, W. Deacon, R. Grisenthwaite, A. Hacquard, and L. Maranget. "Armed Cats: Formal Concurrency Modelling at Arm." In: *ACM Transactions on Programming Languages and Systems* 43.2 (2021), 8:1–8:54. DOI: 10.1145/3458926.

[10] NVIDIA Corporation. *Parallel Thread Execution ISA*. Version 9.3. 2026. URL: https://docs.nvidia.com/cuda/archive/13.3.1/parallel-thread-execution/index.html (visited on 09/03/2026).

[11] LLVM Project. *User Guide for AMDGPU Backend*. Version latest. 2026. URL: https://rocm.docs.amd.com/projects/llvm-project/en/latest/LLVM/llvm/html/AMDGPUUsage.html (visited on 09/03/2026).

[12] A. Goens, S. Chakraborty, S. Sarkar, S. Agarwal, N. Oswald, and V. Nagarajan. "Compound Memory Models." In: *Proceedings of the ACM on Programming Languages* 7.PLDI, 153 (2023), pp. 1145–1168. DOI: 10.1145/3591267.

[13] J. Alglave, L. Maranget, S. Sarkar, and P. Sewell. "Litmus: Running Tests against Hardware." In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2011)*. Vol. 6605. Lecture Notes in Computer Science. Springer, 2011, pp. 41–44. DOI: 10.1007/978-3-642-19835-9_5.

[14] NVIDIA Corporation. *CUDA Programming Guide: Programming Model*. Version 13.3. 2026. URL: https://docs.nvidia.com/cuda/archive/13.3.0/cuda-programming-guide/01-introduction/programming-model.html (visited on 09/05/2026).

[15] L. Fusco, M. Khalilov, M. Chrapek, G. Chukkapalli, T. Schulthess, and T. Hoefler. *Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip*. arXiv:2408.11556v2, 26 August 2024. 2024. arXiv: 2408.11556v2 [cs.DC].

[16] NVIDIA Corporation. *NVIDIA Grace Hopper Superchip Architecture Whitepaper*. 2023. URL: https://www.aspsys.com/wp-content/uploads/2023/09/nvidia-grace-hopper-cpu-whitepaper.pdf (visited on 09/12/2026).

[17] NVIDIA Corporation. *CUDA Programming Guide: Unified and System Memory*. Version 13.3. 2026. URL: https://docs.nvidia.com/cuda/archive/13.3.0/cuda-programming-guide/02-basics/understanding-memory.html (visited on 09/13/2026).

[18] Advanced Micro Devices. *HIP Documentation: Unified Memory Management*. 2026. URL: https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_runtime_api/memory_management/unified_memory.html (visited on 09/13/2026).

[19] Advanced Micro Devices. *HIP Documentation: Coherence Control*. 2026. URL: https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_runtime_api/memory_management/coherence_control.html (visited on 09/03/2026).

[20] NVIDIA Corporation. *CUDA Programming Guide: Unified Memory*. Version 13.3. 2026. URL: https://docs.nvidia.com/cuda/archive/13.3.0/cuda-programming-guide/04-special-topics/unified-memory.html (visited on 10/01/2026).

[21] J. Wahlgren, G. Schieffer, R. Shi, E. A. León, R. Pearce, M. Gokhale, and I. Peng. "Dissecting CPU-GPU Unified Physical Memory on AMD MI300A APUs." In: *2025 IEEE International Symposium on Workload Characterization (IISWC)*. Read as arXiv:2508.12743v1. IEEE, 2025, pp. 368–380. DOI: 10.1109/IISWC66894.2025.00038.

[22] NVIDIA Corporation. *NVIDIA Puts Grace Blackwell on Every Desk and at Every AI Developer's Fingertips*. Jan. 2025. URL: https://nvidianews.nvidia.com/news/nvidia-puts-grace-blackwell-on-every-desk-and-at-every-ai-developers-fingertips (visited on 09/12/2026).

[23] A. Skende. *NVIDIA GB10 SoC: AI Supercomputer On Your Desk*. Presentation at Hot Chips 2025. 2025. URL: https://hc2025.hotchips.org/assets/program/conference/day2/21_nvidia_skende_final.pdf (visited on 09/21/2026).

[24] Advanced Micro Devices. *Introducing AMD CDNA 3 Architecture*. 2025. URL: https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf (visited on 09/03/2026).

[25] D. Das Sharma, R. Blankenship, and D. S. Berger. "An Introduction to the Compute Express Link (CXL) Interconnect." In: *ACM Computing Surveys* 56.11, 290 (2024), pp. 1–37. DOI: 10.1145/3669900.

[26] D. Lustig, S. Sahasrabuddhe, and O. Giroux. "A Formal Analysis of the NVIDIA PTX Memory Consistency Model." In: *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. 2019, pp. 257–270. DOI: 10.1145/3297858.3304043.

[27] S. V. Adve and K. Gharachorloo. "Shared Memory Consistency Models: A Tutorial." In: *Computer* 29.12 (1996), pp. 66–76. DOI: 10.1109/2.546611.

[28] L. Lamport. "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs." In: *IEEE Transactions on Computers* C-28.9 (1979), pp. 690–691. DOI: 10.1109/TC.1979.1675439.

[29] A. Q. Zhang, A. Goens, N. Oswald, T. Grosser, D. Sorin, and V. Nagarajan. "PipeGen: Automated Transformation of a Single-Core Pipeline into a Multicore Pipeline for a Given Memory Consistency Model." In: *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques (PACT '24)*. 2024, pp. 1–13. DOI: 10.1145/3656019.3676889.

[30] J. Suarez. *Thread Synchronization Examples*. Arm Learning Paths, "Explore Thread Synchronization in the Arm Memory Model," Examples 1 and 3: message passing without barriers and with release/acquire instructions. 2026. URL: https://learn.arm.com/learning-paths/servers-and-cloud-computing/memory_consistency/examples/ (visited on 09/24/2026).

[31] J. Alglave and L. Maranget. *A `diy` "Seven" tutorial*. Version 7.58. Manual of the herdtools7 suite. Feb. 12, 2025. URL: http://diy.inria.fr/doc/ (visited on 09/03/2026).

[32] J. Alglave, L. Maranget, and M. Tautschnig. "Herding Cats: Modelling, Simulation, Testing, and Data Mining for Weak Memory." In: *ACM Transactions on Programming Languages and Systems* 36.2 (2014), 7:1–7:74. DOI: 10.1145/2627752.

[33] J. Wickerson, M. Batty, T. Sorensen, and G. A. Constantinides. "Automatically Comparing Memory Consistency Models." In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17)*. 2017, pp. 190–204. DOI: 10.1145/3009837.3009838.

[34] L. Maranget, J. Alglave, et al. *herdtools7*. Version 7.58+1. Git repository, CeCILL-B licence (LICENSE.txt); upstream master inspected at commit 745a9c54 (7 September 2026). 2026. URL: https://github.com/herd/herdtools7 (visited on 09/13/2026).

[35] J. Alglave and P. Cousot. *Syntax and analytic semantics of LISA*. 2016. arXiv: 1608.06583.

[36] J. Alglave, M. Batty, A. F. Donaldson, G. Gopalakrishnan, J. Ketema, D. Poetzl, T. Sorensen, and J. Wickerson. "GPU Concurrency: Weak Behaviours and Programming Assumptions." In: *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '15)*. 2015, pp. 577–591. DOI: 10.1145/2694344.2694391.

[37] S. Srivastava. "Testing Memory Models of Heterogeneous CPU-GPU Systems." UC Santa Cruz Electronic Theses and Dissertations. Master of Science thesis. University of California, Santa Cruz, 2024. URL: https://escholarship.org/uc/item/5hm7q1jv (visited on 09/03/2026).

[38] S. Bagchi. *hetero-cuda-litmus: A Heterogeneous CPU-GPU Litmus Test Framework for NVIDIA GPUs*. Git repository, inspected at commit 06607ac (28 April 2026); no licence file. 2026. URL: https://github.com/sohambagchi/hetero-cuda-litmus (visited on 09/03/2026).

[39] D. Lustig, A. Wright, A. Papakonstantinou, and O. Giroux. "Automated Synthesis of Comprehensive Memory Model Litmus Test Suites." In: *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '17)*. 2017, pp. 661–675. DOI: 10.1145/3037697.3037723.

[40] D. Iorga, A. F. Donaldson, T. Sorensen, and J. Wickerson. "The Semantics of Shared Memory in Intel CPU/FPGA Systems." In: *Proceedings of the ACM on Programming Languages* 5.OOPSLA, 120 (2021), 120:1–120:28. DOI: 10.1145/3485497.

[41] R. Levine, M. Cho, D. McKee, A. Quinn, and T. Sorensen. "GPUHarbor: Testing GPU Memory Consistency at Large (Experience Paper)." In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. 2023, pp. 779–791. DOI: 10.1145/3597926.3598095.

[42] R. Levine, T. Guo, M. Cho, A. Baker, R. Levien, D. Neto, A. Quinn, and T. Sorensen. "MC Mutants: Evaluating and Improving Testing for Memory Consistency Specifications." In: *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '23)*. 2023, pp. 473–488. DOI: 10.1145/3575693.3575750.

[43] T. Melissaris, M. Markakis, K. Shaw, and M. Martonosi. "PerpLE: Improving the Speed and Effectiveness of Memory Consistency Testing." In: *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '20)*. 2020, pp. 329–341. DOI: 10.1109/MICRO50266.2020.00037.

[44] T. Sorensen and A. F. Donaldson. "Exposing Errors Related to Weak Memory in GPU Applications." In: *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '16)*. 2016, pp. 100–113. DOI: 10.1145/2908080.2908114.

[45] J. Kirkham, T. Sorensen, E. Tureci, and M. Martonosi. "Foundations of Empirical Memory Consistency Testing." In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA, 226 (2020), 226:1–226:29. DOI: 10.1145/3428294.

[46] W. Vithanage and O. Liljedahl. *When a Barrier Does Not Block: The Pitfalls of Partial Order*. Arm Community blog. Sept. 15, 2025. URL: https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/when-a-barrier-does-not-block-the-pitfalls-of-partial-order (visited on 09/20/2026).

[47] L. Maranget, S. Sarkar, and P. Sewell. *A Tutorial Introduction to the ARM and POWER Relaxed Memory Models*. Revision 120, 10 October 2012. 2012. URL: https://www.cl.cam.ac.uk/~pes20/ppc-supplemental/test7.pdf (visited on 09/25/2026).

[48] NVIDIA Corporation. *CUDA Core Compute Libraries (CCCL)*. Component libcudacxx (libcu++), read at tag v2.8.1; files cuda_ptx_generated.h, fence.h, and the order and scope constants in cuda/std/__atomic/order.h and scopes.h. 2026. URL: https://github.com/NVIDIA/cccl/tree/v2.8.1 (visited on 09/03/2026).

[49] Advanced Micro Devices. *HIP device headers: amd_hip_atomic.h and amd_device_functions.h*. Git repository ROCm/clr, read at commit 4ea70abd. 2026. URL: https://github.com/ROCm/clr/tree/4ea70abdbcbae416f17a4f10921cafa926cd31fd/hipamd/include/hip/amd_detail (visited on 09/03/2026).

[50] Arm Limited. *Arm A-profile A64 Instruction Set Architecture*. Document DDI 0602, release 2022-12; instructions LDAPR, STLR and DMB. 2022. URL: https://developer.arm.com/documentation/ddi0602/2022-12/ (visited on 10/01/2026).

[51] LLVM Project. *Clang Language Extensions*. Version Clang 22.0.0git. 2026. URL: https://rocm.docs.amd.com/projects/llvm-project/en/latest/LLVM/clang/html/LanguageExtensions.html (visited on 09/03/2026).

[52] LLVM Project. *Clang builtin definitions: Builtins.td*. Version LLVM 23.1.0. Section HIP atomic builtins. 2026. URL: https://github.com/llvm/llvm-project/blob/llvmorg-23.1.0/clang/include/clang/Basic/Builtins.td (visited on 09/20/2026).

[53] L. Zhang, M. Wahib, H. Zhang, and S. Matsuoka. "A Study of Single and Multi-device Synchronization Methods in Nvidia GPUs." In: *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. Read as arXiv:2004.05371. IEEE, 2020, pp. 483–493. DOI: 10.1109/IPDPS47924.2020.00057.

[54] R. Levine, A. Goens, and T. Sorensen. *cuda-litmus*. Git repository, inspected at commit d107bb56 (15 April 2026); no licence file. 2026. URL: https://github.com/reeselevine/cuda-litmus (visited on 09/03/2026).

[55] NVIDIA Corporation. *CUDA Programming Guide: CUDA C++ Execution Model*. Version 13.3. 2026. URL: https://docs.nvidia.com/cuda/archive/13.3.0/cuda-programming-guide/05-appendices/cuda-cpp-execution-model.html (visited on 10/01/2026).

[56] R. Levine. *webgpu-litmus*. Git repository, Apache-2.0 licence; read at commit c7234af7. 2026. URL: https://github.com/reeselevine/webgpu-litmus (visited on 09/03/2026).

[57] S. Vigna. *splitmix64.c*. 2015. URL: https://prng.di.unimi.it/splitmix64.c (visited on 09/03/2026).

[58] Vast.ai. *Vast.ai*. 2026. URL: https://vast.ai/ (visited on 10/02/2026).

[59] Vultr. *NVIDIA GH200 Grace Hopper Superchip*. 2026. URL: https://www.vultr.com/products/cloud-gpu/nvidia-gh200/ (visited on 10/02/2026).

[60] L. Diachin. *DOE Exascale Computing Project. Update for the September ASCAC Meeting*. Slide 6, "HIP on Aurora – Bringing HIP to Intel GPUs", project PI Brice Videau. Sept. 28, 2023. URL: https://science.osti.gov/-/media/ascr/ascac/pdf/meetings/202309/ASCAC-ECP-Update-Sep-2023-rev5.pdf (visited on 09/14/2026).

[61] A. Raad, L. Maranget, and V. Vafeiadis. "Extending Intel-x86 Consistency and Persistency: Formalising the Semantics of Intel-x86 Memory Types and Non-temporal Stores." In: *Proceedings of the ACM on Programming Languages* 6.POPL, 22 (2022). DOI: 10.1145/3498683.